

6BUIS020W

Business Information Systems

Project Report

Project Title: **StudyWest**

Prepared by: **Ryan Rahman**

Supervisor: **Mohammed Shah**

**This report is submitted in partial fulfilment of the requirements for the
BSc (Hons)
Business Information Systems**

School of Computer Science & Engineering

University of Westminster

30th April 2026

Declaration

**This report has been prepared based on my own work. Where other published and unpublished
source**

materials have been used, these have been acknowledged in references.

Gen AI usage statement: I used ChatGPT to support my work by improving the flow, clarity, and grammar of written sections throughout the report. It helped refine wording and structure so that my ideas were communicated more clearly, while ensuring the final content remained based on my own understanding and interpretation. I also used Figma AI to assist in developing and refining functionality-related aspects of my design and code, particularly to explore possible implementation approaches. All outputs generated by these tools were critically reviewed, edited, and integrated by me to ensure accuracy, appropriateness, and alignment with the project requirements.

**Word Count: 12,250
Student Name: Ryan Rahman
Date of Submission: 30/04/2026**

Table of Contents

Project Overview.....	
1. The Purpose of Project.....	8
2. The Scope of the work.....	8
2.a The current situation.....	9
2.b Context of the work.....	6
2.c Work Partitioning.....	7
2.d Competing Products/Related Work.....	8
3. Product Scenarios.....	12
3.a Individual Product Scenarios.....	13
4. Stakeholders.....	10
4b. Customer.....	10
4c. Hands on Users.....	10
5. Constraints.....	11
5a. Current System Implementation Environment.....	11
Constraint 1 – Web-Based Platform.....	11
Constraint 2 – Internet Connectivity.....	11
Constraint 3 – Cloud-Based Backend Infrastructure.....	12
Constraint 4 – Institutional and Legal Compliance.....	12
Partner Application 1 – Authentication and Database Service.....	12
Partner Application 2 – Web Browsers.....	13
II Requirements.....	
6 Product Use Cases.....	13
6a Use Case Diagrams.....	13
6b Individual Product Use Cases.....	14
7. Functional & Non Functional Requirements.....	16
7.3 Requirements Traceability Matrix.....	18
8. Data Requirements.....	20
8a. Safety-Ethical-Critical Requirements.....	20
9. Security Requirements.....	21
9a. Access Requirements.....	21
9b. Integrity Requirements.....	22
9c. Privacy Requirements.....	22
9d. Accessibility Requirements.....	22
10. Legal Requirements.....	23
10a. Compliance Requirements.....	23
III Design.....	
11. Proposed Software Structure.....	25
11a. Class Diagrams.....	25
11b. Dynamic Model.....	26

11c System Model and Decomposition.....	26
11d. Persistent Data Management.....	27
11e. Access Control and Security.....	28
12. User Interface.....	29
12a. Structured Heuristic Usability Evaluation.....	35
IV Implementation.....	
13. Coding.....	36
13a. Technology Selection and Justification.....	36
13b. Key Code Implementations.....	38
Profile Page Code Explanation.....	38
13c. Reflection and Skills Development.....	40
14. Features to Be Tested/Not to be Tested:.....	41
14a. Features to be tested.....	41
14b. Features Not to be tested.....	41
14c. Prototype Limitations vs. Production Vision (Analysis).....	42
15. Test Cases.....	42
15a. Testing Summary and Critical Evaluation.....	44
15b. Usability Evaluation.....	44
15c. Improvement and Future Development.....	45
V Project Summary.....	
16. Delivering Sustainability.....	45
17. Conclusions and Open Issues.....	45
References.....	47
Appendix.....	50
Appendix A: Survey Questions and Results.....	50
Appendix B: Access Control Implementation.....	51

Abstract

StudyWest is a prototype web platform designed to improve how University of Westminster students share peer-generated study materials. Students currently rely on informal tools such as WhatsApp, Teams, email, and cloud links, which makes resources difficult to find, inconsistently accessible, and difficult to moderate.

The project aimed to design and implement a controlled peer-sharing platform with university email registration, structured upload, search and filtering, ratings, reporting, and administrator moderation. The prototype was built using React, TypeScript, Tailwind CSS, and shadcn/ui. A Firebase-based backend architecture was designed for future development, but the submitted artefact uses mock data and client-side state to demonstrate the main workflows.

The prototype demonstrates the core student and administrator journeys, including registration, login, upload, moderation, search, rating, and reporting. Testing showed that most Must-Have requirements were demonstrated at interface level. However, persistent storage, real email verification, server-side security rules, and production-grade file storage remain outside the current implementation.

Overall, StudyWest shows that a moderated student-to-student resource-sharing platform is feasible and useful within the Westminster context, while also identifying the technical and governance work required before real deployment.

Acknowledgements

I would like to thank my supervisor, Mohammed Shah, for his consistent guidance, feedback, and support throughout this project. His comments helped me develop the system design, clarify the implementation scope, and improve the structure of the final report.

I am also grateful to the University of Westminster and the School of Computer Science and Engineering for providing the academic framework for this project.

Finally, I would like to thank the students who completed the survey during the proposal stage. Their responses helped ground the project in real user needs and directly shaped the core system requirements.

List of figures

Figure 1. Survey Results: Problems with current study material sharing methods

Figure 2. StudyWest Work Context Diagram

Figure 3. Use Case Diagram (Student and Administrator)

Figure 4. UML Class Diagram (StudyWest Object Model)

Figure 5. Sequence Diagram (Upload, Report, Moderation Workflow)

Figure 6. StudyWest Login Page

Figure 7. StudyWest Registration Page

Figure 8. StudyWest Student Dashboard

Figure 9. StudyWest Student Profile Page

Figure 10. StudyWest Email Verification Page

Figure 11. StudyWest Administrator Login Page

Figure 12. StudyWest Administrator Dashboard

Figure 13. StudyWest Home Page

List of Tables

Table 1. Competing Products Comparison

Table 2. Functional and Non-Functional Requirements

Table 3. Requirement Traceability Matrix

Table 4. Data Risk and Mitigation Table

Table 5. Frontend Framework Comparison

Table 6. UI Framework and Supporting Libraries

Table 7. Novel vs Adapted Code Classification

Table 8. Features to Be Tested

Table 9. Features Not to be tested

Table 10. Prototype Limitations vs Production Vision

Table 11. Test Execution Log and Results Summary

Table 12. Heuristic Usability Evaluation

Project Overview

1. The Purpose of the Project

StudyWest addresses a specific and unmet need in the University of Westminster learning environment: there is no structured platform for student-to-student sharing of revision notes, summaries, and other peer-generated academic resources. In practice, students rely on fragmented channels such as WhatsApp, Teams, email, and shared cloud links, which makes resources difficult to locate, inconsistently accessible, and largely unmoderated.

The aim of the project was to design and prototype a moderated web-based platform that supports verified student access, structured upload and categorisation, search and filtering, and administrator review of shared content. StudyWest was not intended to replace Blackboard or manage official teaching materials; rather, it was designed as a controlled peer-sharing layer that improves discoverability and accountability. The final artefact should therefore be evaluated as a frontend-led proof-of-concept with a defined production architecture, not as a fully deployed institutional system.

Goals of the Project

The goals of this project are:

- Demonstrate university email-domain validation during student registration, restricting access to verified @my.westminster.ac.uk accounts.
- Enable structured upload of study materials with title, description, course, and module categorisation, with all uploads entering a Pending moderation state before publication.
- Demonstrate an administrator moderation workflow supporting approval, removal, and restoration of uploaded content using a four-state model: Pending, Approved, Flagged, and Removed.
- Prototype role-based access control separating student and administrator functionality at the routing layer.
- Implement keyword search and multi-criteria filtering to allow students to locate materials by course, module, and file type.
- Evaluate core workflows through functional test cases linked to requirements via a Requirement Traceability Matrix.

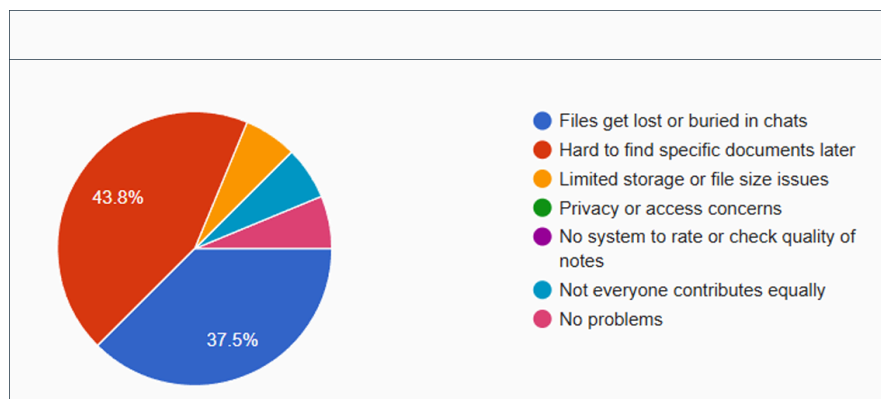
1.a The User Business or Background

The StudyWest system operates within the context of the University of Westminster, a large higher education institution with a diverse student population and a strong emphasis on digital learning. Students regularly engage with online platforms such as virtual learning environments, file-sharing tools, and academic databases as part of their studies. However, existing systems primarily support lecturer-to-student content delivery rather than peer-to-peer knowledge sharing. StudyWest is positioned to complement this environment by enabling students to share

resources collaboratively, improving accessibility to revision materials and supporting independent learning. The system must therefore align with institutional requirements around academic integrity, data protection, and usability, while also meeting student expectations for fast, intuitive, and reliable digital platforms.

2. The Scope of the work

2.a The current situation



At present, University of Westminster students share peer-generated study materials through informal channels including WhatsApp group chats, Microsoft Teams, email attachments, Google Drive, and OneDrive links. These tools support quick communication but are not designed to function as structured academic repositories.

Survey evidence from 16 student responses suggested two main issues: 43.8% found it difficult to relocate shared resources, while 37.5% said files became buried in chat histories. The sample was small and results should be treated as directional rather than statistically representative, but the consistency of these two themes across the majority of respondents provides sufficient justification for a prototype focused on discoverability and organisation.

Although some modules on Blackboard provide lecturer-uploaded materials, there is no structured platform for students to share peer-generated revision resources with each other. This creates several practical problems:

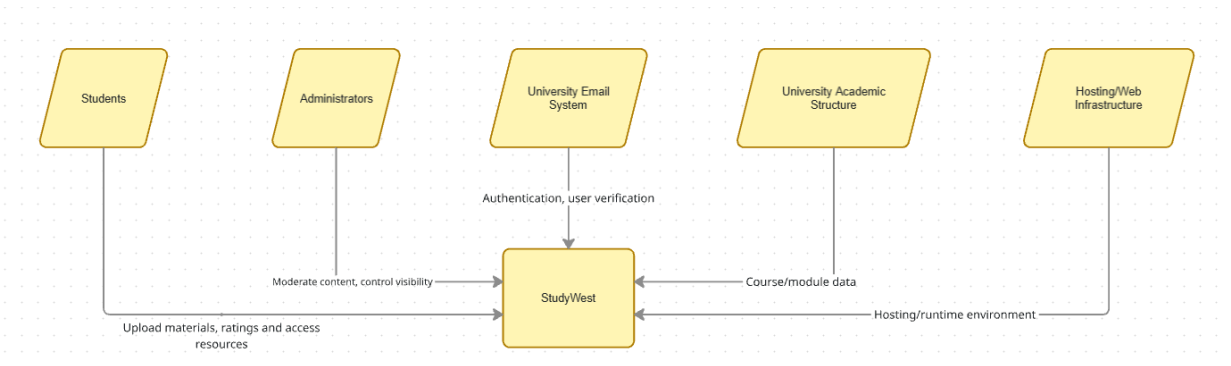
- study materials are fragmented across multiple informal platforms;
- files become buried in chat histories and are difficult to retrieve;
- there is no moderation or quality control over shared content;
- students with access to different group chats receive unequal access to resources;
- there is no search capability to locate materials by course or module.

These problems directly informed the core features of StudyWest: keyword search, course and module categorisation, persistent access to uploaded materials, peer ratings, content reporting, and administrator moderation.

2.b Context of the work

To determine the boundaries of StudyWest's system, a work context diagram was developed (Figure 2). The system interacts with five main external entities, each introducing specific dependencies on authentication, data structure, or infrastructure:

- Students (Primary Users)
- Administrators (Moderation Authority)
- University Email System
- University Academic Structure (Courses, Modules)
- Hosting/Web Infrastructure



The University Email System creates a dependency on institutional authentication services, meaning access control is externally enforced rather than fully managed within StudyWest. The Academic Course Structure constrains the system to existing institutional module hierarchies, ensuring uploaded materials are correctly categorised. Students and Administrators introduce a separation between content creation and governance, with students uploading and rating materials and administrators controlling their visibility.

By establishing this context, StudyWest avoids scope creep and ensures the system remains within a defined boundary: it does not attempt to replace Blackboard, manage official academic records, or extend beyond peer-shared study materials.

2.c Work partitioning (Business Events)

The process of StudyWest's work is represented by a series of events triggered by the real world. Each event relates to something happening externally, which triggers a system response.

Event 1: Student Registers

- **Input:** Registration information (name, university email, password)
- **Output:** Verification email sent or account access granted
- **System behaviour:** Validates input and transitions the user from unauthenticated to pending/verified state. The system checks the university email domain, generates a verification code, and activates the account upon successful verification.

Event 2: Student Uploads Study Material

- **Input:** Study material file, title, description, selected module
- **Output:** Study material saved with status "Pending"
- **System behaviour:** Introduces a state transition from user-generated input to a moderated content pipeline. All uploads enter a controlled review state before publication. A StudyMaterial object is created and queued for administrator review.

Event 3: Student Reports Study Material

- **Input:** Reason for reporting
- **Output:** Material marked as flagged; administrator alerted
- **System behaviour:** Updates the moderation state of a content item and triggers an administrative workflow for review and governance. Moderation statistics are updated accordingly.

Event 4: Administrator Reviews Study Material

- **Input:** Study material pending or flagged for reporting
- **Output:** Study material marked as Approved or Removed
- **System behaviour:** Represents a controlled decision node in the moderation workflow where system state is finalised based on administrative validation. Moderation statistics are updated.

Event 5: Student Searches for Study Materials

- **Input:** Search criteria or selected module
- **Output:** List of approved study materials
- **System behaviour:** Operates on a filtered dataset of approved resources, enforcing access control and ensuring only validated content is returned.

2.d Competing Products/Related Work

Several existing platforms were reviewed to assess whether they already address the need for structured, institution-specific peer-to-peer sharing of study materials. None fully do.

Blackboard is designed for lecturer-to-student delivery of official content. It provides strong institutional control but does not support student-led knowledge sharing or peer-generated revision resources. This gap is well-documented in collaborative learning literature, where institution-managed systems are noted to prioritise delivery over peer exchange (Dillenbourg, 1999).

Google Drive and **OneDrive** allow flexible file sharing but rely on informal folder structures with no academic metadata, categorisation, or governance. Resources become disorganised quickly, and discoverability degrades over time — directly matching the problems identified in the survey, where 43.8% of students reported difficulty relocating previously shared resources.

WhatsApp and **Microsoft Teams** are fast communication tools, not knowledge repositories. Files shared in message threads are difficult to retrieve and provide no moderation, version control, or search capability. The survey confirmed this: 37.5% of respondents said resources became buried in chat histories.

StuDocu is the closest commercial comparator, supporting uploads, search, and peer access to academic materials. However, it is not institution-specific, paywalls some content, and lacks alignment with Westminster module structures. It has also been criticised for hosting copyrighted and assessment-related content, precisely the governance failure that StudyWest's moderation model is designed to prevent.

Existing tools solve separate parts of the problem: WhatsApp and Teams support fast communication, Drive and OneDrive support file storage, and StuDocu supports academic document discovery. However, none combine Westminster-only access, course/module categorisation, peer ratings, and administrator moderation. This combination defines StudyWest's practical contribution.

Platform	Strength	Limitation for StudyWest problem
Blackboard	Official and institution-controlled	Does not support student-led sharing
WhatsApp/Teams	Fast communication	Files get buried and are hard to moderate
Google Drive/OneDrive	Easy file storage	Weak academic categorisation and governance
StuDocu/Course Hero	Large study-resource repositories	Not Westminster-specific and may involve paywalls

3. Product Scenarios

Product scenarios describe how the end user will utilise StudyWest when the system is completely implemented. Written in the form of stories, they focus more on the usage of the system rather than the technology involved. Product scenarios allow non-technical individuals to understand the operation of the system in a real-world environment. They also help reveal how the system behaves under realistic usage conditions, including trust, moderation, and user interaction challenges.

3.a Individual Product Scenarios

Scenario 1: Student Registration and Access

Ryan is a first-year Business Information Systems student at the University of Westminster. During revision week, he hears from a fellow student about StudyWest. He visits the homepage, clicks Register, and enters his name, university email, and password. The system checks whether the email belongs to the official Westminster domain.

After submitting the form, Ryan is taken to a verification page where he enters the six-digit code. Once verified, his account becomes active. This step ensures only authenticated university members can access the platform, reducing the risk of external misuse or unauthorised content access.

Scenario 2: Uploading Study Material

Aisha is a second-year student who has prepared study notes for the Database Systems course. She logs in to StudyWest, navigates to Upload Material, and submits her PDF with a title and module selection. StudyWest displays a confirmation message informing her that the notes are awaiting approval. In the background, the material is tagged as Pending and the Administrator is notified. The moderation queue ensures content is reviewed before becoming publicly accessible.

Scenario 3: Admin Moderation

Mohammed, an Administrator, opens the admin dashboard and sees a list of pending and flagged materials. He reviews a submission that has been reported for potentially reproducing lecture slides without permission. Assessing the content, Mohammed determines the material constitutes original student notes rather than copied material and clicks Approve. Had the content reproduced copyrighted slides, he would have selected Remove, triggering the Notice and Takedown workflow. This scenario illustrates the judgment-based nature of moderation and the importance of administrator training and clear upload declaration policies.

Scenario 4: Finding and Getting Materials

Daniel is preparing for his Programming Fundamentals examination. He logs in, searches by module, and browses a list of approved materials. He selects a resource with a high peer rating, downloads it, and later returns to rate it positively because it clarified key concepts. Daniel's experience illustrates the core value of the platform: peer-rated materials surface useful content efficiently, supporting the kind of student-to-student knowledge exchange that Dillenbourg (1999) identifies as central to collaborative learning.

Together, these four scenarios demonstrate the core workflows of the StudyWest platform: verified registration and access control, structured content upload, administrator-led moderation, and peer-driven content discovery — each directly addressing a functional requirement established in Section 7.

4. Stakeholders

StudyWest has three primary stakeholder groups: students, administrators, and the University of Westminster as the institutional owner.

4a. The Client

The University of Westminster is treated as the main institutional stakeholder. Even though StudyWest is only a student prototype, university expectations around academic integrity, data protection, and educational value shaped the access-control, moderation, and compliance requirements.

4b. The Customer

Students are both the primary customers and the hands-on users. They upload study materials, search and download resources, submit ratings, and report inappropriate content. Their needs — fast discoverability, ease of access, and trust in content quality — directly informed the core functional requirements, including search, categorisation, peer ratings, and moderation feedback. Because students are already familiar with digital platforms, the system prioritises task efficiency and clarity over extensive onboarding.

4c. Hands On Users

Administrators are responsible for content moderation: reviewing pending and flagged submissions, approving or removing materials, and monitoring platform activity. Their requirements defined the four-state content model (Pending, Approved, Flagged, Removed), the moderation workflow, and the separation of administrative routes from student access. Speed and accuracy are critical — moderation decisions directly affect what content other students can access.

These two user groups have partially competing interests. **Students** benefit from fast content availability, while administrators require time to properly review submissions for quality and policy compliance. The Pending state was designed specifically to manage this tension — holding content from public view until reviewed, rather than defaulting to instant publication or full manual gatekeeping of every item.

Indirect Stakeholders

Lecturers have an indirect but significant stake in the platform. Although they do not use StudyWest directly, the quality and accuracy of peer-shared materials within their modules affects student learning outcomes and academic integrity standards. Poorly moderated content could misrepresent module topics, reinforce misconceptions, or facilitate assessment misconduct. Their presence in the stakeholder landscape reinforces the need for strong administrator moderation, clear upload declaration policies, and an accessible Notice and Takedown process.

5. Constraints

These constraints define the technical and organisational limits within which StudyWest was designed and implemented. They influenced key decisions about platform type, backend architecture, access control, and legal compliance.

5a. Current System Implementation Environment

Constraint 1 – Web-Based Platform

Description: The StudyWest system operates as a browser-based web application accessible through modern internet browsers.

Rationale: Students and administrators use a wide range of personal and institution-owned devices. A web-based solution ensures universal access without requiring software installation.

Fit Criterion: The system must function correctly on the latest versions of Google Chrome, Microsoft Edge, and Safari without additional software. Prototype testing showed users could navigate from the Home page to module-specific materials in approximately three steps, indicating efficient navigation.

Constraint 2 – Internet Connectivity

Description: The system requires an active internet connection for authentication, file uploads, file downloads, and content moderation.

Rationale: StudyWest relies on cloud-based services for data storage and user authentication, both of which require real-time connectivity.

Fit Criterion: Core features must operate reliably with an active connection, with clear feedback provided if connectivity is lost.

Constraint 3 – Cloud-Based Backend Infrastructure

Description: The proposed production architecture for StudyWest uses a cloud-hosted backend service (Firebase) for authentication, database storage, and file hosting.

Rationale: A managed cloud backend reduces development complexity, improves scalability, and ensures secure handling of user data and uploaded materials. Firebase was selected due to its integrated Authentication, Firestore database, and Storage services, which map directly to the platform's core requirements.

Fit Criterion: In the submitted prototype, authentication, storage, and database operations are simulated using mock state and in-memory data to allow user flow testing without live backend

infrastructure. Manual prototype testing showed that the simulated workflows operated without critical interface failures. However, because the backend was not implemented, reliability, security, and persistence cannot be fully validated at this stage.

Constraint 4 – Institutional and Legal Compliance

Description: The system must comply with University of Westminster IT policies and UK GDPR data protection regulations.

Rationale: StudyWest stores user accounts and academic materials, requiring secure and ethical handling of personal and educational data.

Fit Criterion: Access to user data must be restricted to authorised users, with role-based controls enforced for administrative functions. This was demonstrated through protected routes and role separation in the prototype.

5b. Partner or Collaborative Applications

Partner Application 1 – Authentication and Database Service

Description: The system integrates with a third-party backend service for user authentication and database management.

Rationale: Reliable authentication and secure data storage are essential components that require robust infrastructure.

Integration Constraint: The system must follow the API structure, security rules, and operational limits defined by the backend service.

Fit Criterion: Users must be able to register, log in, and maintain authenticated sessions without data inconsistencies or authentication failures. In prototype testing, authentication flows (registration, login, and session persistence using mock/managed backend logic) functioned consistently under normal usage conditions, with no critical failures observed.

Partner Application 2 – Web Browsers

Description: The system depends on modern browser technologies such as JavaScript execution, file upload APIs, and local storage.

Rationale: All system functionality is executed client-side within the browser environment.

Integration Constraint: The interface must degrade gracefully if certain browser features are unavailable or restricted.

Fit Criterion: Core functionality must operate consistently across supported browsers without layout distortion or script errors. In prototype testing, the system functioned as expected across modern browsers (Chrome, Edge, Safari), with no major layout or JavaScript execution issues observed in standard desktop environments.

II Requirements

This section defines the core functional and non-functional requirements of the StudyWest system, based on stakeholder needs, user scenarios, and the project proposal.

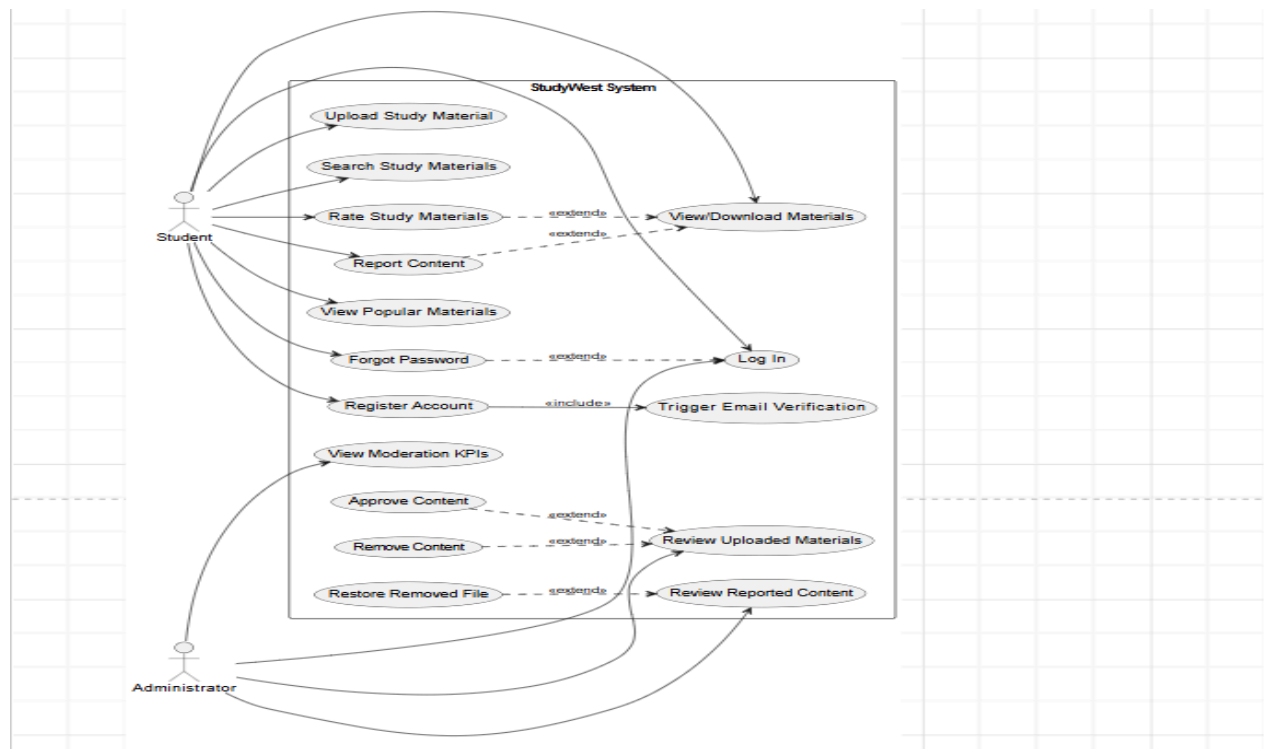
6. Product Use Cases

The StudyWest system enables students and administrators to upload, access, and moderate academic resources. Use cases describe how each actor interacts with the system and define the functional boundaries of the platform.

Requirements for StudyWest were elicited using a structured requirements-engineering approach based on Sommerville (2016), combining stakeholder identification, student survey data, and scenario walkthroughs to ensure that functional requirements were grounded in real user behaviour and institutional context.

6a. Use Case Diagrams

Figure 3 illustrates the primary actors and their interactions with StudyWest, showing the system boundary, available actions per actor, and relationships between use cases (including include and extend dependencies).



The diagram uses include relationships to represent mandatory sub-flows — for example, Register includes Trigger Email Verification, as content upload is only available to verified users. Extend relationships represent optional actions — for example, Report Content extends View/Download Materials, as reporting is an optional action available when viewing a resource.

Actors:

- **Students** – registers an account, logs in, uploads materials, searches, views/downloads, rates, reports, and views popular materials.
- **Administrators** – logs in, reviews uploaded materials, approves or removes content, reviews reported content, restores removed files, and views moderation KPIs.

Use Cases included in the system boundary:

For Students:

- Register Account
- Trigger Email Verification
- Log In
- Forgot Password
- Upload Study Material
- Search Study Materials
- View/Download Materials
- Rate Study Materials
- Report Content
- View Popular Materials

For Administrators:

- Log In
- Review Uploaded Materials
- Approve Content
- Remove Content
- Review Reported Content
- Restore Removed File
- View Moderation KPIs

Use Cases outside the system boundary:

- Creating academic content externally (e.g., Word, PDF)
- Offline teaching or classroom activities

6b Individual Product Use Cases

Below are the detailed use cases for the core student and administrator interactions shown in the diagram.

Use Case 1: Register Account

- **Actor:** Student
- **Description:** A student creates a new StudyWest account.
- **Preconditions:** None
- **Steps:** 1. Student selects "Register". 2. System prompts for required details. 3. Student submits registration form. 4. System triggers email verification. 5. Student verifies email to activate account.
- **Postconditions:** A verified student account is created.

Use Case 2: Forgot Password

- **Actor:** Student
- **Description:** A student resets their account password if they are unable to log in.
- **Preconditions:** Student has a registered account.
- **Steps:** 1. Student selects "Forgot Password" on the login page. 2. System prompts for the registered email address. 3. Student submit their email. 4. System sends a password reset link to the email. 5. Student follows the link and enters a new password. 6. System updates the password and confirms the reset.
- **Postconditions:** Student can log in using the new password.

Use Case 3: Upload Study Material

- **Actor:** Student
- **Description:** A student uploads an academic file to StudyWest.
- **Preconditions:** Student is logged in; file is in an acceptable format.
- **Steps:** 1. Student selects "Upload Material". 2. System prompts for file selection. 3. Student chooses a file and confirms. 4. System validates file type and size. 5. System stores the file and marks it as "Pending".
- **Postconditions:** Material is uploaded and available for admin moderation.

Use Case 4: Search and View/Download Materials

- **Actor:** Student
- **Description:** A student searches for study materials and downloads a selected file.
- **Preconditions:** Student is logged in.
- **Steps:** 1. Student enters keywords or applies filters. 2. System displays matching materials. 3. Student selects a resource. 4. Student views or downloads the file. 5. System records the download for analytics.
- **Postconditions:** Student has accessed the selected resource.

Use Case 5: Rate Study Material

- **Actor:** Student
- **Description:** A Student rates a study resource.
- **Preconditions:** Student is logged in.
- **Steps:** 1. Student opens a resource. 2. System records the rating and updates the displayed average; in the current prototype this is computed client-side and is not persisted across sessions. . 3. System updates the resource's average rating.
- **Postconditions:** Rating is stored and reflected in the resource's score.

Use Case 6: Report Content

- **Actor:** Student
- **Preconditions:** Student is logged in.
- **Steps:** 1. Student selects "Report". 2. System flags the resource. 3. System notifies administrators.
- **Postconditions:** Resource is marked for admin review.

Use Case 7: Moderate Content

- **Actor:** Administrator
- **Preconditions:** Admin is logged in.
- **Steps:** 1. Admin views pending or flagged materials. 2. Admin opens a resource. 3. Admin chooses to approve, remove, or restore. 4. System updates the resource status. 5. System optionally notifies the reporting student.
- **Postconditions:** Resource status is updated according to the admin's decision.

Use Case 8: View Moderation KPIs

- **Actor:** Administrator
- **Preconditions:** Admin is logged in.
- **Steps:** 1. Admin opens the moderation dashboard. 2. System displays KPIs and statistics.
- **Postconditions:** Admin has access to up-to-date moderation insights.

7. Functional & Non-Functional Requirements

This section outlines the capabilities of the StudyWest system. Requirements were prioritised using the MoSCoW method (Must have, Should have, Could have, Won't have), ensuring that the Minimum Viable Product (MVP) focuses on the core student-to-student sharing loop while deferring secondary aesthetic enhancements.

Requirements table

ID	Requirement	Description (Summary)	Owner	Priority (MoSCoW)	Type
FR-01	Student Registration	Students must register and login using a valid @my.westminster.ac.uk email and complete email verification	Students	Must Have	Functional
FR-02	User Role Management	Platform functionality must be restricted based on user role (Student or Admin)	IT Department	Must Have	Functional
FR-03	File Upload	Students must upload study materials (PDF, Word, images)	Students	Must Have	Functional
FR-04	File Access	Students must view and download materials uploaded by others	Students	Must Have	Functional
FR-05	File Categorisation	Materials must be categorised by course and module	Students	Must Have	Functional
FR-06	Search Functionality	System must support search by keyword, course, or module	Students	Must Have	Functional
FR-07	Rating System	Students can rate materials (1–5 stars, one rating per user)	Students	Should Have	Functional
FR-08	Reporting Content	Students must be able to report inappropriate content	Students	Must Have	Functional

FR-09	Administrative Controls	Admins must approve, remove, and restore content	Admins	Must Have	Functional
FR-10	Feedback and Confirmation	Users must receive feedback for actions (upload, login, errors)	Students	Should Have	Functional
FR-11	Popular Materials Display	System should highlight popular or highly rated materials	Students	Should Have	Functional
NFR-01	User Interface	Interface must be user-friendly, responsive, and clear	UI/UX	Must Have	Non-Functional
NFR-02	Security	Prevent unauthorised access and enforce role-based control	IT Department	Must Have	Non-Functional
NFR-03	Performance	Search results must return within 500ms; page load < 3 seconds	IT Department	Should Have	Non-Functional
NFR-04	Maintainability	System must use modular architecture for easy updates	IT Department	Should Have	Non-Functional

7.3 Requirement Traceability Matrix (RTM)

The Requirement Traceability Matrix links each requirement to implementation evidence and testing outcomes. Because the submitted artefact is a prototype, some outcomes reflect interface-level validation rather than full backend enforcement. To make this clear, results are labelled using implementation maturity categories:

- Passed – implemented and demonstrated in the prototype;
- Passed (Prototype) – function demonstrated, but without full backend persistence;
- Passed (Mock) – behaviour validated using mock data;
- Passed (Simulated) – workflow demonstrated without live infrastructure;
- Partial Pass – feature works only in limited cases;

- Fail – expected behaviour not achieved consistently.

This allows the matrix to show not only whether a requirement was demonstrated, but also how completely it was implemented.

Where test coverage is absent, this reflects prototype scope limitations rather than missing requirements, and these areas are identified for future validation.

RTM Table

Req ID	Requirement	Implementation Evidence	Test Case	Status
FR-01	Student registration using Westminster email and verification step	RegisterPage.tsx, VerifyEmailPage.tsx , validation utilities	TC003, TC004	Passed Simulated
FR-02	Role-based access for students and administrators	LoginPage.tsx, AdminLoginPage.tsx, ProtectedRoute logic	TC001, TC011, TC015	Passed Mock
FR-03	Upload study material	UploadModal.tsx, DashboardPage.tsx , mock material state	TC005, TC014	Passed Prototype
FR-04	View/download approved materials	MaterialCard.tsx, MaterialViewer.tsx	TC006	Passed Prototype
FR-05	Categorise materials by course and module	Material metadata fields, dashboard filters	TC006	Passed
FR-06	Search by keyword, course, module, and file type	Dashboard filtering logic	TC007, TC008, TC016	Partial Pass
FR-07	Rating system	Rating component in MaterialCard.tsx	TC010	Passed Mock
FR-08	Report inappropriate content	Report action and flagged admin view	TC009	Passed Prototype
FR-09	Admin moderation controls	AdminPage.tsx approve/remove/restore handlers	TC009	Passed Prototype
FR-10	User feedback and confirmation	Toast notifications and validation messages	TC001, TC003, TC005	Passed

FR-11	Popular/highly rated materials	Dashboard sorting and popular materials section	TC012	Passed Mock
NFR-01	Usable and responsive interface	Responsive layout and shadcn/ui components	TC012	Passed
NFR-02	Access control and security	Protected routes and role checks	TC011, TC015	Passed Mock
NFR-03	Acceptable prototype performance	Manual navigation/search testing	TC013	Partial Pass
NFR-04	Maintainable modular structure	React component-based structure	Design review	Achieved Design-Level

Not formally testable at prototype stage; performance acceptable on small mock dataset only.

8. Data Requirements

The StudyWest system relies on structured data entities that support uploading, searching, rating, reporting, and moderating academic resources. These form the foundation of the system's domain model.

User

Represents any registered individual.

Attributes: UserID, Name, University Email, Role (Student/Administrator), Verification Status, Account Status

Resource (Study Material)

Represents academic content uploaded by students.

Attributes: ResourceID, Title, Description, File URL, Course, Module, Upload Date, Uploaded By (UserID), Status (Pending/Approved/Flagged/Removed), Average Rating, Download Count

Rating

Represents a student's quality assessment of a resource.

Attributes: RatingID, ResourceID, UserID, Rating Value (1–5)

Report

Represents a student's complaint about a resource.

Attributes: ReportID, ResourceID, Reported By (UserID), Reason, Date Reported, Review Status

ModerationLog

Represents administrator actions for audit and accountability in a full system implementation.

Attributes: LogID, AdminID, ResourceID, Action (Approve/Remove/Restore), Timestamp, Notes

Key relationships:

- A User can upload multiple Resources (Students only).
- A Resource can receive multiple Ratings and multiple Reports.
- An Administrator can update the status of multiple Resources.
- Each moderation action generates one ModerationLog entry.

8a. Safety-Ethical-Critical Requirements

Because StudyWest handles student-uploaded content and institutional email addresses, the main risks are content-related, reflecting established principles of academic integrity (ICAI, 2021).

Risk	Description	Mitigation
Copyright violation	Students upload textbooks, lecture slides, or paid resources	Notice and Takedown process; upload declaration; admin review
Academic misconduct	Assessment answers or solution guides shared	Moderation queue; admin removal; clear acceptable use policy
Inappropriate content	Offensive or non-academic material uploaded	Pending state prevents immediate publication; admin approval required
Malicious reporting	Legitimate resources falsely flagged to disrupt access	Admin reviews all reports before action; restore function available
Data protection breach	Email addresses and files mishandled under UK GDPR	Restricted access; encrypted storage; RBAC enforcement

Fit Criteria:

- Administrators must be able to remove or restore flagged content through the moderation dashboard, with changes reflected immediately.
- All moderation actions must be recorded in an audit log reviewable by administrators.
- Reported content must appear in the admin review queue within two seconds of submission.
- The system must support GDPR safeguards through secure storage, encrypted communication, and restricted access permissions.

9. Security Requirements

Security in StudyWest should be understood at two levels: the controls demonstrated within the submitted prototype and the additional enforcement mechanisms required for production. The prototype includes interface-level restrictions, validation logic, and role separation. However, because the current version does not rely on a live backend with enforceable authorisation rules, security features represent a partial implementation of the intended access-control model rather than a complete end-to-end security architecture.

9a. Access Requirements

- The platform is restricted to registered users with a verified university email.
- The system implements role-based access control, where students can upload, download, rate, and report resources, while administrators have permissions to approve, delete, and restore content.
- Administrative routes are protected to prevent unauthorised access and ensure only authorised users can perform moderation actions.

Fit Criterion:

All authenticated users must be able to access student features after login, while unauthorised attempts to admin routes must be blocked and redirected or return an access denied response.

9b. Integrity Requirements

- The system prevents duplicate ratings by enforcing one rating per user per resource.
- Uploaded files are validated before submission, rejecting unsupported file types and files exceeding the maximum size limit (e.g. 10MB).
- The content status is consistent and restricted to a controlled set: Pending, Approved, Flagged, and Removed.
- The database is protected from data corruption and accidental deletion.

Fit Criterion:

Duplicate ratings must be blocked, invalid file uploads must be rejected, content status values must remain within the defined set, and data must remain consistent without loss or corruption during normal system operation.

9c. Privacy Requirements

- User personal data such as email, must remain confidential and only accessible to authenticated users and admins.
- Data must be securely stored and protected from unauthorised access in accordance with UK GDPR requirements.
- Users must be informed that uploaded study materials are visible to other registered users.

Fit Criterion:

Personal data must not be publicly accessible, must be restricted to authorised users, and must be stored securely in compliance with UK GDPR standards.

9d. Accessibility Requirements

The system must provide a consistent and usable interface across standard desktop environments. Core pages must render correctly on Chrome, Edge, and Safari at standard desktop resolution (1920×1080), with readable text, clear navigation labels, and sufficient colour contrast to support users with colour vision deficiencies, in line with WCAG 2.1 Level AA contrast guidance. Interface elements must be clearly labelled to support keyboard navigation and screen reader compatibility where possible.

Fit Criterion:

All core pages (login, dashboard, upload, profile) must display readable text at standard resolution with no layout failures across supported browsers. Colour contrast ratios should meet the WCAG 2.1 AA minimum of 4.5:1 for normal text. In the current prototype, accessibility was considered at design level through colour scheme choices and labelling conventions. Formal WCAG compliance testing was not conducted and should be prioritised in future development.

10. Legal Requirements

10a. Compliance Requirements

StudyWest must comply with UK GDPR, the Data Protection Act 2018, and relevant university acceptable-use policies. This is necessary because the system stores student account data and hosts user-generated academic content.

For data protection, the system should apply four core principles:

- Data minimisation: collect only the data needed for authentication, access control, and moderation.
- Purpose limitation: use account and content data only for platform operation.
- Storage limitation: retain personal data only while the account remains active or moderation records are required.
- Security: restrict access to authorised users and protect stored data through appropriate technical controls.

Users should be informed during registration that their university email, profile data, and uploaded materials will be stored and made visible to other verified users where relevant.

From a copyright perspective, students must not upload copyrighted textbooks, paid materials, or lecture content without permission. A production version of StudyWest would therefore

require a clear notice-and-takedown process, moderation records, and an account deletion process to support the right to erasure.

In the current prototype, these requirements are acknowledged at design level, but full legal compliance would depend on backend enforcement, documented policies, and institutional governance.

III Design

11. Proposed Software Structure

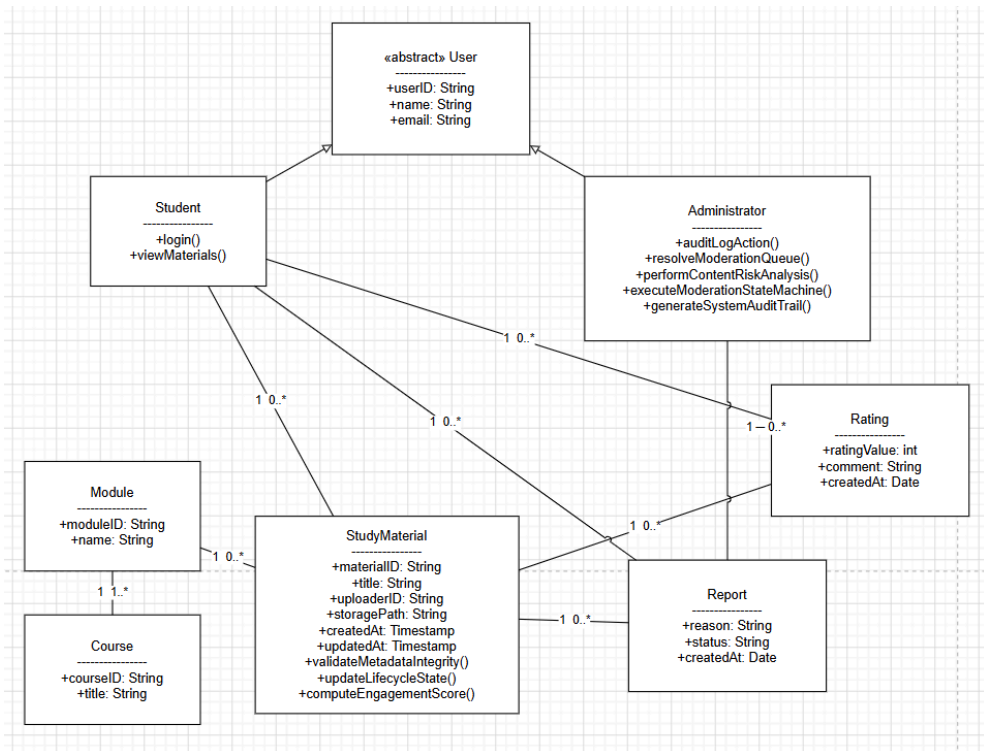
11a. Class Diagrams

The StudyWest class diagram (Figure 4) provides a static structural overview and demonstrates how object-oriented principles support clarity, modularity, and scalability. At the centre is an abstract User class containing shared attributes (userID, name, email) and authentication operations (login(), logout()), from which Student and Administrator inherit specialised behaviour.

The class diagram uses inheritance to model the domain conceptually: students and administrators share common user attributes but have different responsibilities.

In the implemented React prototype, this is represented more simply through typed user objects and role-based routing rather than full class inheritance. This avoids duplication while enabling role-specific method definitions. This design choice ensures consistency in authentication handling while supporting extensibility, as additional user roles could be introduced without restructuring the core system model. A key trade-off is tighter coupling between classes — changes to the User authentication model require corresponding updates in both subclasses. A microservices approach would improve independent deployability, but was rejected due to significant infrastructure overhead unsuitable for a rapid prototype.

The StudyMaterial class includes explicit lifecycle methods (updateLifecycleState(), computeEngagementScore()) to represent the moderation state machine. This was chosen deliberately: distinguishing between Pending, Approved, Flagged, and Removed states allows the system to enforce different access permissions per state, support administrator triage workflows, and maintain a complete audit trail — none of which a simple boolean field could support. This improves traceability and aligns system behaviour with real-world moderation workflows, where content must transition through clearly defined states rather than binary



approval logic. The Administrator class supports content moderation through operations including `auditLogAction()`, `resolveModerationQueue()`, and `performContentRiskAnalysis()`.

11b. Dynamic Model

Upload Process

The sequence diagram (Figure 5) captures the complete workflow for uploading, reporting, reviewing, and moderating study materials. A sequence-based model was preferred over a static event-log approach because it provides clearer visibility of state progression over time and improves traceability between individual interactions and system state transitions, which is essential for understanding moderation workflows where state changes depend on prior events.

Upload Process:

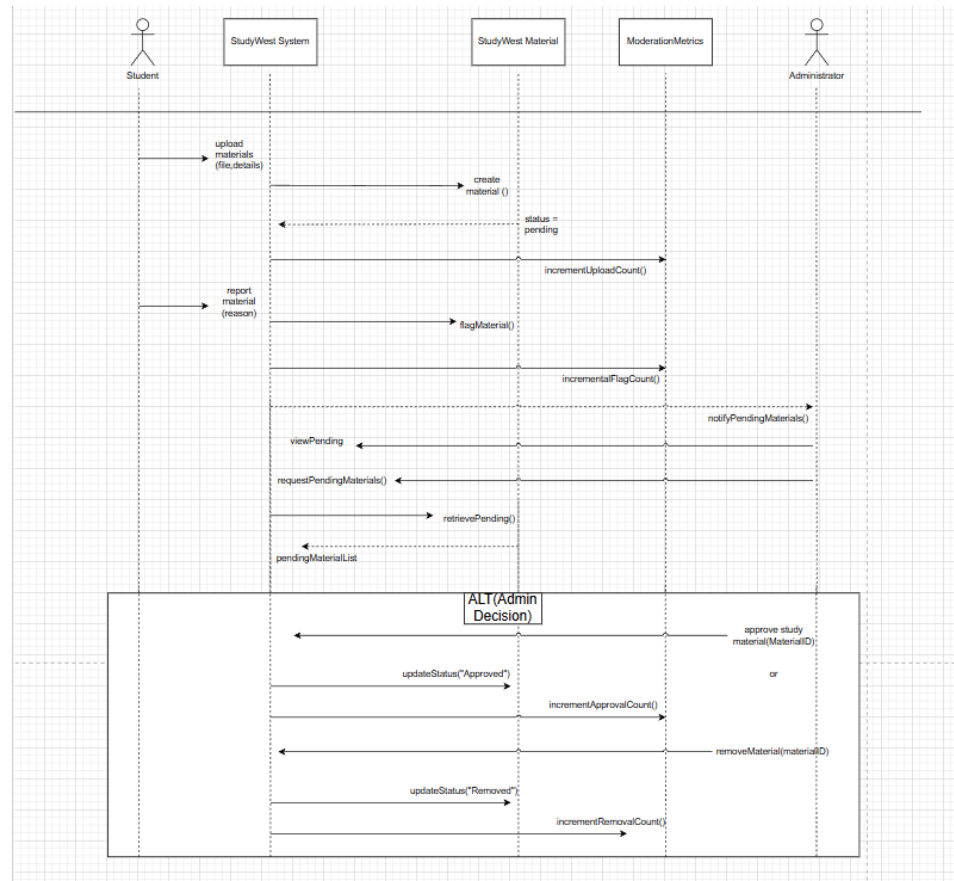
When a student uploads a resource, the material's status is automatically set to Pending, ensuring all content enters a controlled validation state before becoming publicly accessible.

Reporting Process:

When a student submits a report, the system flags the corresponding StudyMaterial object and notifies the administrator. Pending items are made available for review through a filtered administrator view.

Moderation Decision (ALT Fragment):

The diagram includes an ALT fragment representing two possible outcomes: Approval updates the StudyMaterial status to Approved, making it visible to all students; Removal updates the status to Removed, hiding it from student views and recording the action in the ModerationLog. Explicit state transitions (Pending → Approved/Removed) prevent ambiguous content states and ensure consistent moderation logic.



Design Justification

The four-state content lifecycle was chosen because it reflects the moderation process more accurately than a simple approved/not-approved field. A newly uploaded file begins as Pending, becomes Approved if accepted by an administrator, becomes Flagged if reported, and can be moved to Removed if it violates platform rules. This makes the moderation process easier to test and explain, because each state has a clear meaning and controls whether students can access the material.

11c. System Model and Decomposition

The StudyWest system is composed of three complementary models: the Functional Model (Use Case Diagrams), the Object Model (Class Diagram), and the Dynamic Model (Sequence Diagrams). Rather than existing as independent artefacts, these form a traceable design chain where functional requirements are progressively refined into structural and behavioural representations.

For instance, the Upload Study Material use case (Functional Model) maps directly to the StudyMaterial class and its updateLifecycleState() method (Object Model), which is then traced through the sequence diagram's ALT fragment showing the Pending → Approved/Removed transition (Dynamic Model). This chain demonstrates that every system behaviour originates from a stated user requirement.

A key strength of this decomposition is traceability: functional requirements identified at the use-case level are systematically realised in both the object model and dynamic interactions. This ensures system behaviour is derived from user-driven requirements and reflected consistently across static structure and runtime execution. At the implementation level, this maps to a React-based interface layer (user interaction), application logic (handling uploads and retrieval), a document-based data layer (Firestore for StudyMaterial entities), and an authentication layer (Firebase Auth).

11d. Persistent Data Management

Certain data must be retained long-term to support system functionality, including user account information (userID, name, university email, passwordHash), study material metadata (materialID, title, description, upload date, moderation status), ratings, reports, and moderation-related data. This ensures users can reliably access, evaluate, and manage academic resources over time.

In contrast, some data is non-persistent. For example, the six-digit email verification code generated during registration is temporary and discarded once verification is complete.

A NoSQL document database (Firebase Firestore) was selected due to its scalability and suitability for storing document-heavy, semi-structured data such as study materials and user-generated metadata. Its flexible document model aligns with the platform's dynamic data

requirements and avoids the rigidity of fixed relational schemas, where variable attributes (e.g. modules, file types, rating histories) are harder to model (Moniruzzaman and Hossain, 2013). Uploaded files are stored separately in Firebase Storage to maintain efficient querying and lightweight document structures.

In the current prototype, persistence is partially simulated, meaning data is not reliably retained across sessions. This reinforces the system's status as a frontend-driven proof of concept rather than a fully deployed solution.

11e. Access Control and Security

Role-based access control is implemented at the React Router layer using a ProtectedRoute component. Unauthenticated users are redirected to /login, and authenticated users without an administrator role are redirected to /dashboard when attempting to access /admin. The prototype stores the current user role in a mock authentication state using getCurrentUser(), applying route-level checks to separate student and administrator views. Students access dashboard, upload, search, rating, and reporting features; administrator functions are restricted to a separate admin dashboard accessed via AdminLoginPage.tsx.

This implementation follows a role-based access control (RBAC) model and adheres to the principle of least privilege, ensuring users can only access resources required for their role. However, the current implementation relies on client-side enforcement, which is not secure and can be bypassed by a determined user manipulating application state. In a production system, Firebase Security Rules would enforce role-based permissions at the database layer, ensuring that unauthorised access is prevented even through direct API calls and providing robust server-side protection.

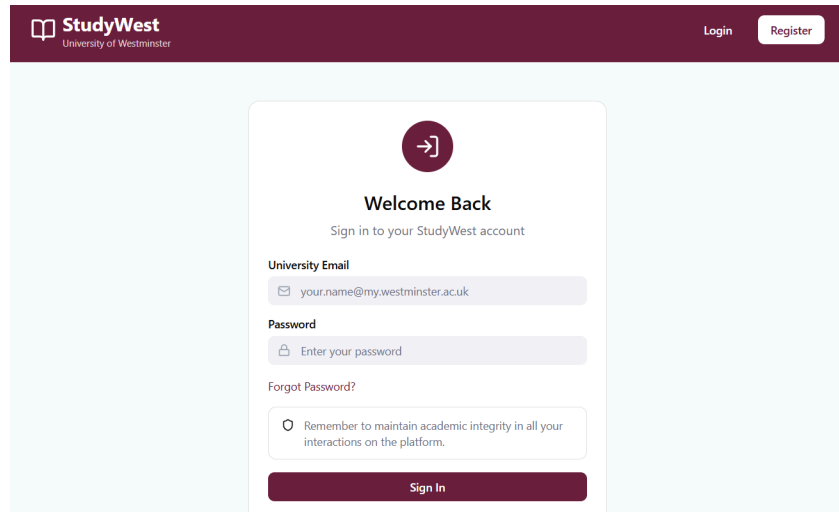
This is the most significant security limitation of the current prototype. Until server-side Firebase Security Rules are implemented, the system should not be treated as secure against a determined user and should not handle real student data.

12. User Interface

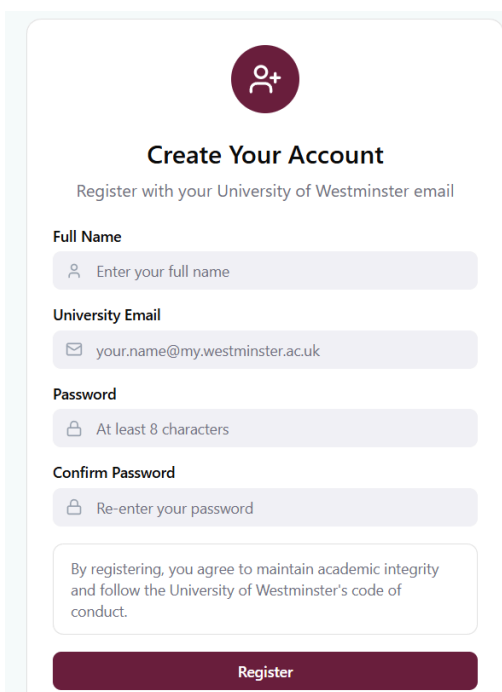
Interface designs were developed in Figma prior to implementation to validate user flows and reduce rework. Design decisions were evaluated against Jakob Nielsen's ten usability heuristics (Nielsen, 1994), and a structured heuristic evaluation was conducted on the completed prototype.

1. Login page (Figure 6)

The login page uses a minimal two-field layout to reduce cognitive load. Placeholder text and inline validation guide input format, reducing the error rate before submission. The registration page enforces Westminster email domain validation and includes a password confirmation field and an academic integrity notice — balancing access control with behavioural expectations. This interface satisfies FR-01 (student registration and login) and NFR-01 (usable and responsive interface), demonstrating that Westminster email validation and clean minimal layout reduce errors before submission.

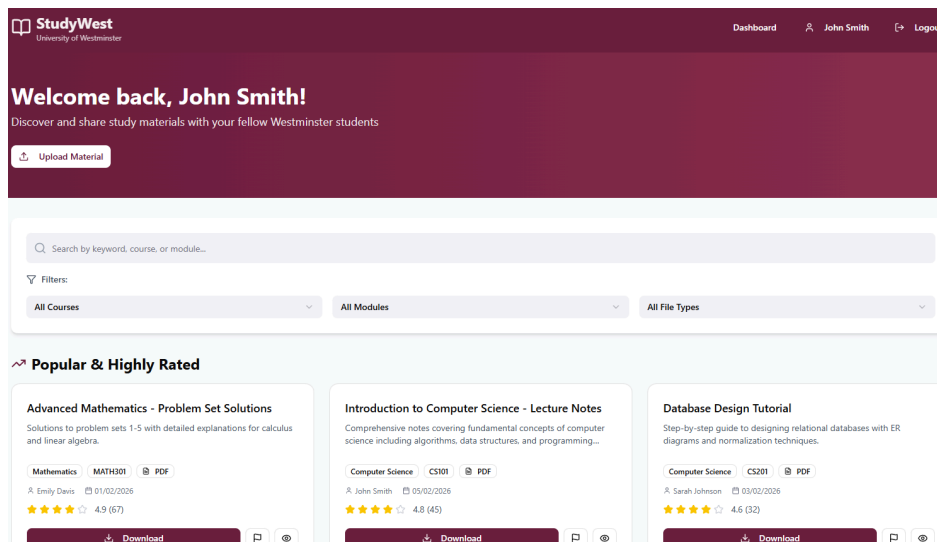


2. Registration page (Figure 7)



In the Registration Page, University of Westminster email domain validation is enforced at the client level, restricting access to institutional users. Field ordering follows a logical top-down input sequence, and password confirmation reduces data entry errors. An academic integrity notice is presented before submission, aligning behavioural expectations with system policy.

3. Dashboard page (Figure 8)

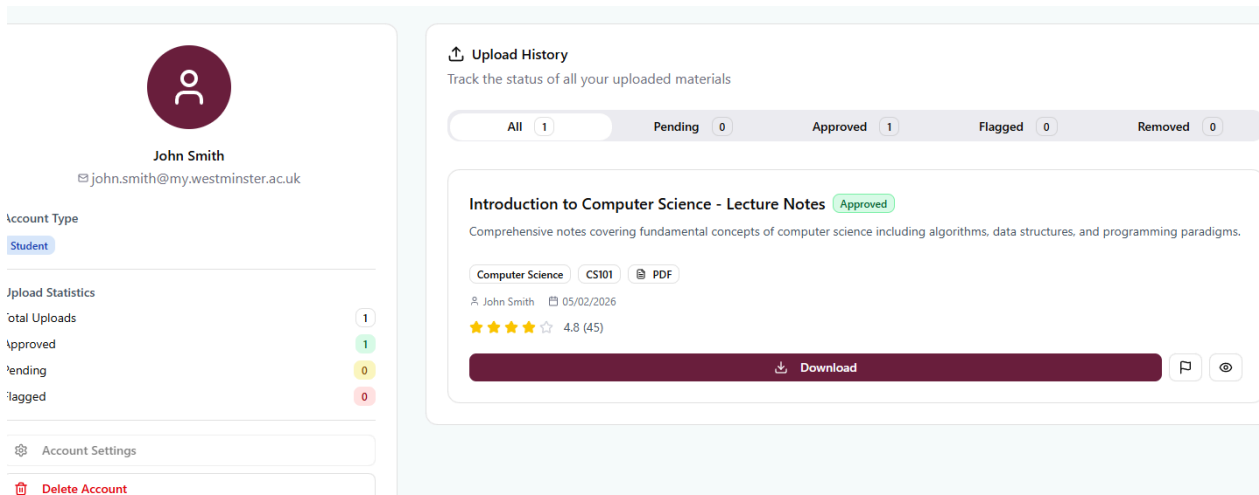


The dashboard implements real-time keyword search and multi-criteria filtering across title, description, course, and module fields. A "Popular & Highly Rated" section ranks approved materials by a weighted combination of average rating and download count. This interface fulfills FR-06 (content search and filtering) and FR-11 (popular materials display).

The most significant limitation identified in testing was filter failure (TC008): combined course

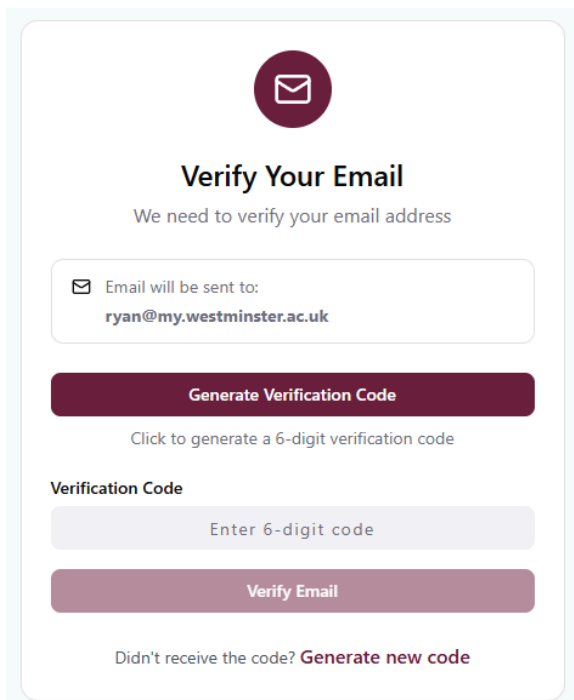
and module filters produced inconsistent results. An empty-state message is displayed when no results are returned, but the underlying filtering logic is unreliable when multiple criteria are applied simultaneously. This would be addressed in a production version through indexed Firestore compound queries rather than client-side array traversal, which limits both accuracy and scalability.

4. Profile page (Figure 9)



The profile page centralises user-specific data, improving visibility of system status and reducing navigation effort. Uploaded materials are displayed alongside their current moderation status, allowing students to track submissions without requiring administrator contact. Filtering by status supports efficient management of multiple uploads, while clear metadata fields support recognition over recall (H6). This interface satisfies FR-03 (file upload visibility) and FR-10 (user feedback and confirmation).

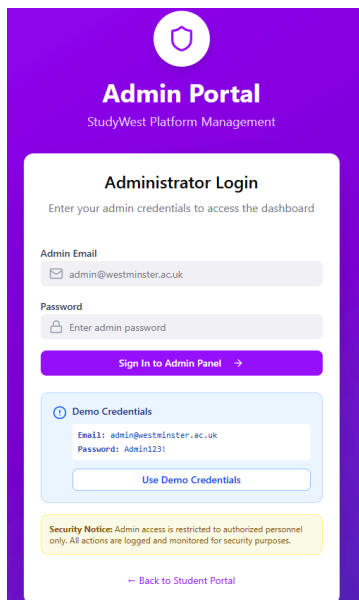
5. Verify email page (Figure 10)



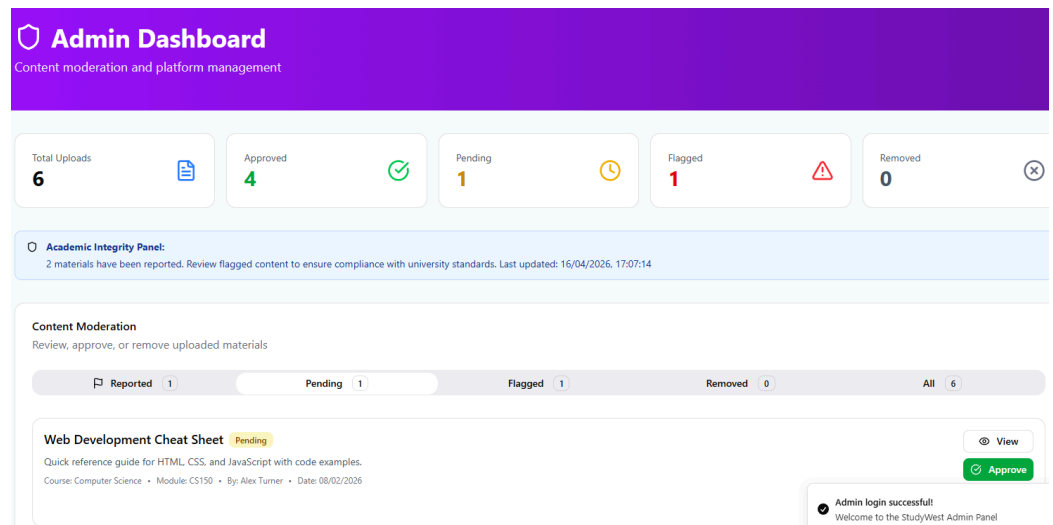
The 'Verify Your Email' page features a maroon envelope icon at the top. Below it, the title 'Verify Your Email' is followed by the instruction 'We need to verify your email address'. A box indicates the email will be sent to 'ryan@my.westminster.ac.uk'. A prominent maroon button labeled 'Generate Verification Code' is present, with a subtext 'Click to generate a 6-digit verification code'. Below this is a 'Verification Code' section with a light grey input field 'Enter 6-digit code' and a maroon 'Verify Email' button. At the bottom, a link 'Generate new code' is provided for users who haven't received the code.

The email verification process supports error prevention by enforcing a required step before account activation, reducing the risk of unauthorised access. The resend function aligns with Nielsen's flexibility and efficiency principle by allowing users to recover from failed delivery without restarting the registration flow. This interface satisfies FR-01 (email verification step) and supports NFR-02 (access restricted to verified users only).

6. Admin Login and Dashboard page (Figures 11-12)



The 'Admin Portal' page has a purple header with a shield icon and the text 'Admin Portal' and 'StudyWest Platform Management'. The 'Administrator Login' section prompts users to enter credentials. It includes fields for 'Admin Email' (pre-filled with 'admin@westminster.ac.uk') and 'Password'. A purple 'Sign In to Admin Panel' button is provided. Below, 'Demo Credentials' are shown with email 'admin@westminster.ac.uk' and password 'Admin123!'. A 'Security Notice' states that admin access is restricted to authorized personnel. A 'Back to Student Portal' link is at the bottom.

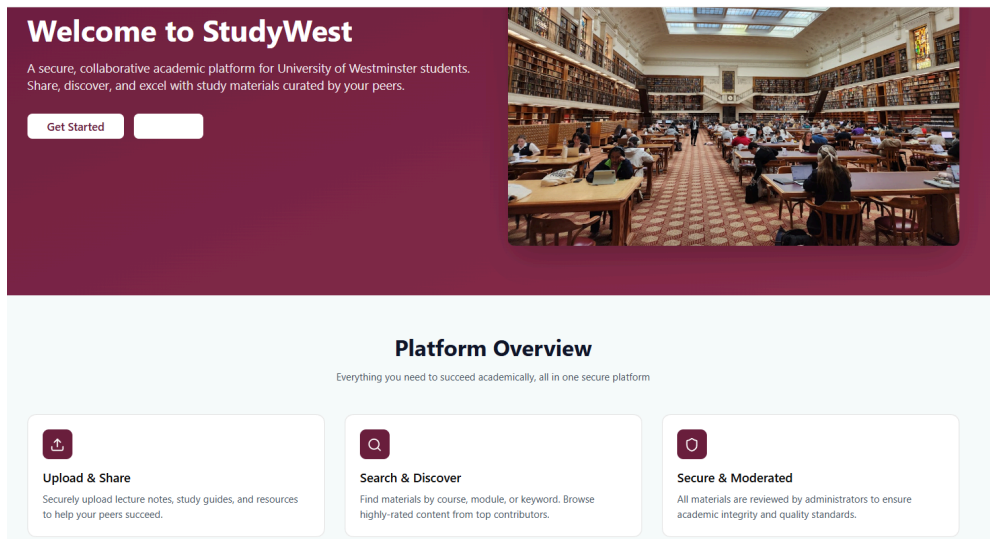


The 'Admin Dashboard' features a purple header with a shield icon, the title 'Admin Dashboard', and the subtitle 'Content moderation and platform management'. It displays five KPI cards: 'Total Uploads' (6), 'Approved' (4), 'Pending' (1), 'Flagged' (1), and 'Removed' (0). Below these is an 'Academic Integrity Panel' alerting that 2 materials have been reported. The 'Content Moderation' section includes a filter bar with tabs for 'Reported' (1), 'Pending' (1), 'Flagged' (1), 'Removed' (0), and 'All' (6). A content item 'Web Development Cheat Sheet' is shown with a 'Pending' status label, a 'View' button, and an 'Approve' button. A success message at the bottom right reads 'Admin login successful! Welcome to the StudyWest Admin Panel'.

The admin login page uses a visually distinct colour scheme and an explicit “Admin Portal” label to prevent role confusion. The admin dashboard presents moderation KPIs followed by a filterable content list, with colour-coded status labels (Pending, Approved, Flagged, Removed)

supporting rapid triage. This interface meets FR-09 (administrative controls) and NFR-02 (role-based security). A key limitation is that moderation actions update client-side state only and are not persisted to a backend. As a result, decisions are lost on page refresh, reducing system reliability and preventing a realistic moderation workflow. In a production system, this would require server-side storage and database integration to ensure persistence and support auditability.

7. Home page (Figure 13)



The public-facing home page presents platform features using a card-based layout, with a clear call-to-action directing users to register or log in. It exists outside the authentication boundary and contains no sensitive data. Following Krug's (2014) principle that users should not have to think unnecessarily, the design reduces the number of steps required to begin using the system and supports efficient onboarding.

Structured Heuristic Usability Evaluation

The prototype was evaluated using Nielsen's usability heuristics rather than formal user testing. This was suitable for identifying obvious design issues within the project timeframe, but it does not provide the same evidence as testing with real students. The main usability issue found was the inconsistent behaviour of combined filters, which could reduce user trust in the search function. Before deployment, the interface should be tested with Westminster students using realistic upload, search, and moderation tasks.

Heuristic	Issue Identified	Severity
H1 – Visibility of system status	Moderation status visible in profile, but upload confirmation toast disappears before users can read the action taken	2 – Minor

H5 – Error prevention	File upload modal does not warn users before submitting unsupported file types; error only appears post-submission	3 – Major
H6 – Recognition over recall	Filter labels ("All Courses", "All Modules") do not indicate that no filter is applied; users must infer this	2 – Minor
H9 – Help users recognise errors	Filter failure (TC008): combined course and module filters give inconsistent results; even when no results are shown, the filtering logic remains unreliable with multiple criteria applied.	3 – Major
H10 – Help and documentation	No onboarding guidance for first-time users; upload declaration appears only at submission stage	2 – Minor

The most significant usability issue identified was the filter failure in TC008, where combined course and module filters returned inconsistent results. Although an empty-state message is present to inform users when no results are found, the filtering logic itself is unreliable when multiple criteria are applied simultaneously. This would be resolved in a production version by replacing client-side filtering with indexed Firestore compound queries.

This evaluation was design-based rather than empirical. No formal user testing was conducted, which is a limitation — usability issues identified through heuristic review may not fully reflect real user behaviour. Formal testing against WCAG accessibility standards would be required before production deployment.

13. Coding

The StudyWest technology stack was selected to support rapid prototyping, modular interface development, and future scalability within the practical time constraints of a final-year project. I made the deliberate decision early in development to prioritise frontend completeness and user-flow validation over full backend implementation, so that the most important student and administrator journeys could be demonstrated clearly. This means the submitted artefact should be evaluated as an architectural and interaction prototype rather than a fully deployed full-stack application.

Interface designs were prototyped using Figma to validate layout and user flows prior to implementation. This reduced rework during development, although design tools do not capture implementation constraints such as state management or backend integration.

Technology Selection and Justification

Frontend Framework Comparison

Criteria	React	Vue	Angular
Learning Curve	Moderate	Easy	Steep
Flexibility	High	Moderate	Low
Ecosystem	Very Strong	Strong	Strong
Community Support	Excellent	Good	Good
Suitability for StudyWest	Excellent	Good	Less suitable
Weighted Score	9.0 / 10	8.0 / 10	7.6 / 10

Each framework was evaluated against five criteria, weighted according to project relevance. Ease of development was weighted highest at 30%, as a single-developer final-year prototype depends heavily on iteration speed — a framework that slows development quickly becomes a practical limitation. Ecosystem strength was weighted at 25%, since StudyWest requires features such as protected routes, modal dialogs, real-time state updates, and dashboard components, all of which have mature solutions in React through libraries such as shadcn/ui and React Router. Flexibility was weighted at 20% to support independent student and administrator workflows without rigid constraints. Community support (15%) and learning curve (10%) were weighted lower, particularly as prior React experience reduced the impact of Vue’s simpler syntax.

Applying these weights, React achieved the highest score (9.0/10), driven by strong ecosystem support and development efficiency. Vue scored 8.0/10, performing well in simplicity but lacking depth for the specific requirements of this project. Angular scored lowest (7.6/10) due to its steeper learning curve and additional architectural complexity, which is less suitable for a single-developer prototype. React was therefore the most appropriate choice.

However, while React offers flexibility and a strong ecosystem, it introduces increased state management complexity at scale, often requiring additional tools such as Zustand or Redux in a production environment.

UI Framework and Supporting Libraries

Tool	Purpose	Justification
Tailwind CSS	Styling	Utility-first approach enabling rapid development and full design control
shadcn/ui	UI Components	Lightweight reusable components aligned with Tailwind

React Router	Navigation	Supports client-side routing and protected routes
Sonner	Notifications	Provides immediate toast feedback for user actions
Figma	Design	Used to prototype and validate UI before implementation
TypeScript	Type Safety	Reduces runtime errors and improves maintainability

Novel vs Adapted Code Classification

Code Area / Feature	Classification	Description
React component structure (Dashboard, Profile, Admin pages)	Novel	Entire page and component architecture designed and implemented specifically for StudyWest requirements
Routing system (React Router setup, protected routes)	Adapted	Based on standard React Router implementation patterns and documentation examples
Form validation logic (registration, login, input handling)	Novel	Custom validation rules and state handling tailored to university email verification and StudyWest workflow
Search and filtering system (Dashboard filtering logic)	Novel	Custom multi-criteria filtering logic built for StudyWest academic categorisation system
Firebase authentication integration	Adapted	Firebase-style auth design — mock Firebase pattern, with full integration planned for production.
UI components (shadcn/ui usage)	Adapted	Built using pre-defined component library structures with custom styling adjustments
Moderation workflow logic (approve / remove / restore)	Novel	Designed specifically for StudyWest academic moderation model
Mock data handling and simulated backend behaviour	Novel	Created to emulate database interactions in absence of full backend implementation

This demonstrates that while external libraries were used, the core logic, workflows, and system design are original.

Key Code Implementations

The following examples illustrate how these concepts are applied in practice through key functional components of the system.

1. Dynamic Search and Filtering Logic (DashboardPage.tsx)

```
// My implementation of real-time search filtering

const filteredMaterials = materials.filter(material => {

  if (material.status !== 'approved') return false;

  const matchesSearch =

    material.title.toLowerCase().includes(searchQuery.toLowerCase()) ||

    material.description.toLowerCase().includes(searchQuery.toLowerCase()) ||

    material.course.toLowerCase().includes(searchQuery.toLowerCase()) ||

    material.module.toLowerCase().includes(searchQuery.toLowerCase());

  const matchesCourse = courseFilter === 'all' || material.course === courseFilter;

  const matchesModule = moduleFilter === 'all' || material.module === moduleFilter;

  const matchesFileType = fileTypeFilter === 'all' || material.fileType === fileTypeFilter;

  return matchesSearch && matchesCourse && matchesModule && matchesFileType;

});
```

This implementation performs client-side keyword search and multi-criteria filtering on an in-memory dataset. The design enables real-time UI updates without server requests, improving responsiveness and user experience.

However, this approach has a clear scalability limitation. Filtering operates with $O(n)$ time complexity because the application must scan the full materials array whenever the user changes the search term or filter criteria. While performant for small datasets, this would degrade significantly at scale. In a production system, this would be replaced with **Firestore compound queries and indexed fields**, improving both performance and accuracy.

2. Real Time Form Validation (Registerpage.tsx)

// My implementation of real-time form validation

```
const handleChange = (field: string, value: string) => {

  setFormData(prev => ({ ...prev, [field]: value }));

  let error = "";

  if (touched[field as keyof typeof touched]) {

    if (field === 'fullName') {

      const validation = validateFullName(value);

      error = validation.error || "";

    } else if (field === 'email') {

      const validation = validateEmail(value);

      error = validation.error || "";

    } else if (field === 'password') {

      const validation = validatePassword(value);

      error = validation.error || "";

    } else if (field === 'confirmPassword') {

      const validation = validatePasswordMatch(formData.password, value);

      error = validation.error || "";

    }

  }

}
```

```
setErrors(prev => ({ ...prev, [field]: error }));  
};
```

The registration form uses real-time validation driven by React state and a touched-state mechanism. This ensures validation feedback is shown only after user interaction, improving usability by avoiding premature error messages.

This increases state complexity. In a production system, validation would be abstracted using schema-based libraries such as Zod or Yup to improve maintainability and reduce duplication.

3. User Specific Data Filtering and State Updates (ProfilePage.tsx)

// My implementation of data filtering and state updates

```
useEffect(() => {  
  if (!currentUser) {  
    navigate('/login');  
    return;  
  }  
}
```

```
const materials = mockMaterials.filter(m => m.uploadedBy === currentUser.fullName);  
setUserMaterials(materials);
```

```
const interval = setInterval(() => {  
  const updatedMaterials = mockMaterials.filter(m => m.uploadedBy ===  
currentUser.fullName);  
  setUserMaterials(updatedMaterials);  
}, 1000);
```

```
return () => clearInterval(interval);
```

```
}, [refreshKey]);
```

This logic filters user-specific data and enforces authentication at the component level. A polling mechanism (`setInterval`) is used to simulate real-time updates in the absence of a backend.

While this worked for prototyping, I realised during development that polling every second was wasteful — it re-ran the filter on every tick regardless of whether any data had changed. This became obvious when I noticed the component re-rendering unnecessarily during manual testing. In a production system, this would be replaced with Firestore's `onSnapshot` listener, which fires only when data actually changes, eliminating the overhead entirely.

4. Moderation State Machine — Approve and Remove Logic (`AdminDashboard.tsx`)

```
// My implementation of the moderation state machine
const handleModerateContent = async (
  materialId: string,
  action: 'approve' | 'remove' | 'restore'
) => {
  const statusMap: Record<string, StudyMaterialStatus> = {
    approve: 'approved',
    remove: 'removed',
    restore: 'approved',
  };

  const newStatus = statusMap[action];
  if (!newStatus) return;

  setMaterials(prev =>
    prev.map(m =>
      m.id === materialId
        ? { ...m, status: newStatus, lastModeratedAt: new Date().toISOString() }
        : m
    )
  );

  setModerationMetrics(prev => ({
    ...prev,
    approved: action === 'approve' || action === 'restore'
      ? prev.approved + 1
      : prev.approved,
    removed: action === 'remove' ? prev.removed + 1 : prev.removed,
    pending: action === 'approve' ? prev.pending - 1 : prev.pending,
  }));
};
```

This function implements a **state machine for moderation actions**, mapping administrator actions to controlled status transitions. The use of a statusMap ensures all valid transitions are explicitly defined and easily extendable.

State updates are handled in a single React state update, which is sufficient for the prototype. However, this is not truly atomic in the database sense because no backend transaction is used. Although, in the current prototype, these updates exist only in client-side state.

In a production system, this logic would require:

- **Server-side validation** to prevent client manipulation
- **Firestore transactions** to avoid race conditions
- **Persistent audit logging (ModerationLog collection)**

Additionally, the absence of a confirmation step for destructive actions represents a usability issue (Nielsen H5 – error prevention), which would need to be addressed before deployment.

Summary

The coding approach prioritised modularity, responsiveness, and rapid development, successfully delivering a functional prototype aligned with system requirements. However, several limitations remain, particularly around scalability, persistence, and security, due to the reliance on client-side logic.

Future development would involve migrating key functionality — including authentication, filtering, and moderation — to backend services, enabling a fully integrated, secure, and scalable system.

The four-state moderation model (Pending, Approved, Flagged, Removed) implemented as a client-side state machine represents a design pattern applicable to any institutional content governance system requiring controlled publication workflows. While the current implementation is frontend-only, the state transition logic and RBAC routing architecture are directly transferable to a production Firebase environment and provide a reusable foundation for similar university platform projects.

Reflection and Skills Development

This project required me to develop several technical skills I had not used before or had only encountered at a surface level in previous modules. The most significant was working with TypeScript in a real project context — specifically writing typed interfaces for complex state objects such as AuthUser and StudyMaterial, and using union types to enforce the controlled status set (Pending, Approved, Flagged, Removed) at compile time rather than relying on runtime checks. This was harder than I expected because TypeScript errors in React components are often indirect, and I spent time learning to read type inference errors properly rather than just suppressing them.

I also learned to implement role-based routing from scratch using React Router's nested route structure and a custom ProtectedRoute component. I had used React Router for basic navigation before but had never built protected routes with conditional role checks, and understanding how to pass requiredRole as a prop and evaluate it against live auth state was new to me.

Working with Firestore's document model was another area of genuine learning. I had used relational databases in earlier modules but designing a NoSQL schema — deciding what to embed in documents versus what to store as separate collections, and understanding why StudyMaterial metadata should stay separate from Firebase Storage file objects — required a different way of thinking about data structure that I had to work through during development.

14. Features to be tested/not to be tested

Testing was conducted using a structured functional evaluation approach, focusing on whether each feature behaves according to its expected output under normal and invalid input conditions. Each test case was evaluated on a pass/fail basis, where a test is considered successful if the system produces the expected result without errors or unintended behaviour.

Testing covered user workflows (registration, login, dashboard interaction), validation logic (form inputs and role checks), and system feedback mechanisms (notifications and navigation). Both valid and invalid inputs were used to assess system robustness and error handling consistency across features.

Equivalence partitioning was applied to form validation testing, grouping inputs into valid Westminster email, invalid format, and missing domain categories to ensure boundary conditions were covered without redundant test cases.

Features to Be Tested:

The following table outlines the key features included in the testing process, what will be tested, and the expected outcomes.

Feature	What Will Be Tested	Expected Result
Student Registration	Full name, university email, password, confirm password, validation	Users can only register with valid Westminster email
Email Verification	Code generation, copy-code button, code validation	Users can generate, enter, and verify a valid 6-digit code
Student Login	Email validation, password field, login success/failure	Successful login redirects students to the dashboard
Admin Login	Admin credentials, invalid attempts, admin navigation	Only valid admin credentials allow admin dashboard access

Dashboard Search	Keyword search by title, description, course, module	Relevant materials appear based on the search query
Dashboard Filters	Course, module, file type filters	Materials update correctly according to selected filters
Upload Material	Opening modal, submitting materials, refreshing list	Newly uploaded materials appear in dashboard and profile
Material Viewer	Opening/closing viewer, displaying material details	Users can correctly view selected study materials
Popular Materials	Display of highly rated materials	Top-rated approved materials appear when no filters applied
Profile Page	User details, upload statistics, history tabs	Users see accurate information and upload history
Report Material	Reporting materials and refreshing list	Reported materials update correctly in the system
Navigation	Links between login, register, dashboard, profile, admin	Users can move between pages without errors
Role-Based Access	Student vs admin permissions	Students cannot access admin pages without admin role
Form Validation	Empty fields, invalid emails, weak/mismatched passwords	Clear error messages appear for invalid input
Toast Notifications	Success/error messages	Appropriate feedback appears after user actions

Features Not to Be Tested:

Some features are excluded because they fall outside the scope of this prototype or rely on simulated data.

Feature	Reason for Exclusion
Real Database Integration	Prototype uses mock data instead of a live backend
Real Email Sending	Verification codes generated internally, not sent via email
Cloud File Storage	Files are not stored in a real cloud environment
Large-Scale Performance Testing	Prototype not designed for high-volume traffic
Security Penetration Testing	Advanced security testing is outside project scope
Multi-Factor Authentication	Advanced authentication not included in prototype

Payment Systems	No payment functionality exists in the platform
API Testing	No external APIs are used in prototype
Server Failure Recovery	No backend server exists in the prototype

Prototype Limitations vs. Production Vision (Analysis)

The following table clarifies the boundary between the current prototype and a production-ready system, directly addressing the assessment requirement to distinguish between simulated and fully implemented features:

Feature	Current Prototype State	Production Target (Post-Submission)
User Authentication	Client-side mock state; domain validation simulated	Firebase Authentication with @my.westminster.ac.uk OAuth or email/password
Email Verification	Code generated in-memory; no actual email sent	Firebase Dynamic Links + NodeMailer / SendGrid API integration
File Storage	File objects held in component state; not persisted	Firebase Storage with file-type/size validation via Cloud Functions
Database	Mock JavaScript arrays in component state	Cloud Firestore with security rules enforcing RBAC
RBAC Enforcement	ProtectedRoute pattern; role checked from mock user object	Role stored in Firestore users/{uid} document; verified server-side via Firebase Security Rules
Rating System	averageRating computed client-side; not persisted across sessions	Firestore transaction to atomically update sum/count fields on StudyMaterial document
Moderation KPIs	Static counters in mock ModerationMetrics object	Real-time Firestore listeners on aggregated counters

15. Test Cases

Testing was conducted using a combination of black-box functional testing and heuristic usability evaluation. The primary goal was to validate the "Traceability Chain" from requirements to the implemented prototype.

Test Execution Log and Results Summary

Test ID	Req ID	Test Scenario	Steps	Expected Result	Actual Result	Outcome	Limitation
TC001	FR-02	Login with valid credentials	Enter valid email and password, then submit	User is redirected to dashboard	Dashboard loaded successfully after login	Passed	Uses mock authentication state
TC002	FR-02	Login with invalid email format	Enter invalid email format and submit	Validation error shown	Error displayed before submission	Passed	Client-side validation only
TC003	FR-01	Registration with valid Westminster email	Enter valid details and submit	User proceeds to verification step	Verification page displayed successfully	Passed (Simulated)	No real email sent
TC004	FR-01	Registration with non-Westminster email	Enter non-Westminster email and submit	Registration blocked	Validation message displayed and submission prevented	Passed	Domain validation is client-side only
TC005	FR-03	Upload study material	Upload file, select module, submit	File saved with Pending status	Material appears with Pending status	Passed (Prototype)	No persistent backend storage
TC006	FR-04 / FR-05	View and categorise material	Open approved item and check metadata	Material is viewable and correctly categorised	Viewer loads and metadata displays correctly	Passed	Based on mock dataset
TC007	FR-06	Basic keyword search	Enter keyword in search bar	Matching materials displayed	Relevant results returned	Passed	Limited dataset size
TC008	FR-06	Combined filtering	Select course and module	Results match selected	Inconsistent / empty results	Failed	Client-side filtering logic is weak

			filters together	filters	returned		
TC009	FR-08 / FR-09	Report and moderate content	Report material, then review in admin dashboard	Material is flagged and reviewable by admin	Flagged material visible in admin workflow	Passed (Prototype)	No backend persistence
TC010	FR-07	Rate study material	Submit rating for a resource	Rating is stored and average updated	Rating displayed correctly in UI	Passed (Mock)	No persistent backend validation
TC011	NFR-02	Access admin route as student	Attempt to open admin route while logged in as student	Access denied / redirected	User redirected to dashboard	Passed	Routing-layer enforcement only
TC012	FR-11 / NFR-01	Popular materials and usability	Open dashboard and review top materials section	Popular materials visible and layout remains clear	Section displayed correctly and navigation remained usable	Passed	Informal usability review only
TC013	NFR-03	Performance on prototype dataset	Perform repeated search and navigation actions	Acceptable response time	Responsive on small dataset	Partial Pass	No benchmarking or scale testing
TC014	FR-03	Upload invalid file type	Attempt to upload a .exe file	Upload rejected with error message	Upload rejected and error shown	Passed	Client-side validation only
TC015	NFR-02	Non-admin attempts moderation action	Log in as student, manually navigate to admin moderation endpoint	Access denied or redirected	Redirected to dashboard	Passed	Routing-layer only
TC016	FR-06	Empty search result feedback	Apply filters that return no results	Empty state message shown	Alert component displayed: "No materials found"	Passed	Client-side only; no backend query optimisation

					matching your search criteria.”		
--	--	--	--	--	---------------------------------------	--	--

Testing Summary and Critical Evaluation

Testing confirmed that the core prototype workflows operate correctly under normal conditions. Registration, login, upload, reporting, moderation, and role-based route protection all behaved as expected during manual functional testing. This provides reasonable confidence that the main student and administrator journeys were implemented correctly at interface level.

The most significant failure was TC008, where combined course and module filters produced inconsistent and incomplete results. This indicates that the current client-side filtering logic is unreliable when multiple criteria are applied simultaneously. In a production system, this would be redesigned using indexed Firestore compound queries rather than in-memory array traversal.

Testing was limited to manual black-box functional evaluation and heuristic usability review. No automated unit tests, integration tests, security tests, or performance benchmarks were conducted. This is a recognised limitation of the prototype: automated unit testing was not implemented due to time and prototype scope. However, unit tests would still be useful for stable client-side logic such as email validation, password validation, and filtering. Full integration and security testing would become meaningful once Firebase Authentication, Firestore, and Storage are implemented. The results validate interface behaviour and user flow logic, but cannot confirm system-level correctness, security, or scalability. These testing types would be prioritised in any production development cycle.

The decision to use equivalence partitioning for form validation testing was deliberate — grouping inputs into valid Westminster email, invalid format, and missing domain categories ensured boundary conditions were covered without producing redundant test cases.

Usability Evaluation

A heuristic review using Nielsen’s principles was carried out. The system performs well in consistency and visibility of system status. However, issues were identified in error prevention (H5), particularly during file upload, where validation messages are not specific enough to guide user correction. This was considered a medium severity usability issue. Additionally, filtering feedback lacks clarity in edge cases, reducing user confidence in system behaviour.

Improvement/Future Development

Future development should focus on converting the prototype into a persistent and secure web application.

The first priority is backend integration, including Firebase Authentication, Firestore, and Firebase Storage, so that users, files, ratings, and moderation actions are stored permanently rather than in client-side state. The second priority is server-side access control, using security

rules to enforce roles beyond the routing layer. The third priority is search and filtering improvement, replacing client-side filtering with indexed backend queries to improve accuracy and scalability.

Additional work should include moderation audit logs, real email verification, account deletion support, and formal usability and accessibility testing against WCAG guidelines. Together, these improvements would move StudyWest from a validated prototype to a credible production-ready system.

V Project Summary

16. Delivering Sustainability

Using Elkington's (1997) Triple Bottom Line framework as a lens, StudyWest's sustainability case is strongest on the social dimension, moderate on the economic dimension, and weakest on the environmental dimension.

StudyWest's strongest sustainability contribution is social rather than environmental. The platform supports educational continuity by preserving peer-generated revision notes and study guides that would otherwise be lost in chats, email threads, or temporary cloud links. This can help future students access useful informal learning materials within the same course and module structure.

Economically, a university-specific platform could reduce student reliance on paid commercial platforms such as StuDocu or Course Hero. However, this benefit depends on moderation effort, hosting costs, and long-term institutional support. Without clear ownership and administrator time, the platform would quickly lose quality and trust.

The environmental benefit is limited. Digital sharing may reduce some printing, but cloud hosting and file storage still consume energy. For this reason, the strongest sustainability argument for StudyWest is not reduced environmental impact, but improved access, reuse, and governance of academic resources.

17. Conclusions and Open Issues

This project successfully delivered a functional prototype demonstrating that moderated peer-to-peer academic resource sharing is both technically feasible and educationally valuable within the University of Westminster context. The consistency between initial requirements, system models, and implemented functionality validates the design approach and requirements elicitation process.

Key Achievements:

- Verified student access control through university email validation
- Functional moderation workflow with role-based permissions
- Intuitive search and categorisation aligned with academic structure
- Demonstration of the main Must-Have requirements at prototype or interface level

Technical Contributions: The project demonstrates effective use of modern frontend technologies, particularly React and TypeScript, within educational project constraints. Firebase was designed as the production backend architecture, although it was not fully implemented in

the submitted prototype. The state machine approach to content moderation and role-based routing architecture provide reusable patterns for similar institutional systems.

Critical Limitations: The most significant constraint is the absence of persistent backend enforcement. While client-side logic demonstrates system behavior, production deployment requires server-side validation, audit logging, and data persistence. Current security controls are interface-level only—a determined user could potentially bypass client-side restrictions.

Open Issues Requiring Resolution:

1. **Backend Integration:** Firebase Authentication, Firestore, and Storage implementation
2. **Security Hardening:** Server-side access control enforcement via security rules
3. **Search Optimisation:** Replace client-side filtering with indexed backend queries
4. **Moderation Audit Trail:** Persistent logging for accountability and governance
5. **Accessibility Compliance:** Formal WCAG evaluation and remediation

Practical Contribution: This project illustrates how a targeted, institution-specific approach to peer resource sharing can address gaps that general-purpose tools leave unfilled, and provides a foundation for further development. The moderation-first approach and academic integrity focus differentiate it from commercial alternatives while maintaining educational value.

Future Development: Converting this prototype into a production system requires addressing persistence, security, and scalability constraints. However, the core workflows, user interface patterns, and system architecture provide a solid foundation for institutional deployment.

The project objective to improve student-to-student sharing of study materials has been achieved at prototype level, with all Must-Have requirements demonstrated through simulated user flows and validated through functional testing.

Testing confirmed that the main prototype workflows operate correctly at interface level. Out of 16 test cases, 14 passed at prototype or interface level, one partially passed, and one failed. The one identified failure was TC008 — combined course and module filtering — where inconsistent results were returned, indicating that client-side filtering is unreliable when multiple criteria are applied simultaneously. This is the clearest development priority for a production version.

References

Figma (2026) Figma design platform. Available at: <https://www.figma.com/> (Accessed: January 2026 to April 2026).

Nielsen, J. (1994) *Usability Engineering*. San Francisco: Morgan Kaufmann. Available at: <https://www.nngroup.com> (Accessed: March 2026).

Sandhu, R.S., Coyne, E.J., Feinstein, H.L. and Youman, C.E. (1996) Role-Based Access Control Models. *IEEE Computer*, 29(2), pp. 38–47. Available at: <https://doi.org/10.1109/2.485845> (Accessed: 16 March 2026).

Ferraiolo, D.F., Barkley, J.F. and Kuhn, D.R. (1999) A Role-Based Access Control Model and Reference Implementation Within a Corporate Intranet. *ACM Transactions on Information and System Security*, 2(1), pp. 34–64. Available at: <https://doi.org/10.1145/300830.300834> (Accessed: 20 March 2026).

Moniruzzaman, A.B.M. and Hossain, S.A. (2013) NoSQL Database: New Era of Databases for Big Data Analytics – Classification, Characteristics and Comparison. Available at: <https://arxiv.org/abs/1307.0191> (Accessed: 28 March 2026).

Sommerville, I. (2016) *Software Engineering*. 10th edn. Boston: Pearson. Available at: <https://www.pearson.com> (Accessed: 18 March 2026).

Kuhn, D.R. (2009) Role-Based Access Controls. Available at: <https://arxiv.org/abs/0903.2171> (Accessed: 25 March 2026).

Ferraiolo, D. and Kuhn, D.R. (1992) Role-Based Access Control. *15th National Computer Security Conference*, Baltimore, pp. 554–563. Available at: <https://csrc.nist.gov/projects/role-based-access-control/rbac-library> (Accessed: 23 March 2026).

Tailwind CSS (2026) Tailwind CSS documentation. Available at: <https://tailwindcss.com/> (Accessed: 20 Feb 2026).

TypeScript (2026) TypeScript documentation. Available at: <https://www.typescriptlang.org/> (Accessed: 5 Feb 2026).

Shadcn UI (2026) Shadcn UI component library. Available at: <https://ui.shadcn.com/> (Accessed: 15 Feb 2026).

Lucide (2026) Lucide icon library. Available at: <https://lucide.dev/> (Accessed: 22 March 2026).

Node.js (2026) Node.js documentation. Available at: <https://nodejs.org/en/docs> (Accessed: 24 March 2026).

Vite (2026) Vite documentation. Available at: <https://vitejs.dev/> (Accessed: 26 March 2026).

University of Westminster (2026) *6BUI5020W Business Computing Final Year Project: tools, templates and resources*. Blackboard VLE. Accessed: December 2025 to April 2026.

Elkington, J. (1997) *Cannibals with Forks: The Triple Bottom Line of 21st Century Business*. Oxford: Capstone. Available at: [Cannibals with Forks: The Triple Bottom Line of 21st Century Business - John Elkington - Google Books](#) (Accessed: 23 April 2026)

Dillenbourg, P. (1999) *What do you mean by collaborative learning?* In: Dillenbourg, P. (ed.) *Collaborative-learning: Cognitive and Computational Approaches*. Oxford: Elsevier, pp. 1–19. Available at: <https://telearn.archives-ouvertes.fr/hal-00190240/document> (Accessed: 23 April 2026).

International Center for Academic Integrity (2021) *The Fundamental Values of Academic Integrity*. 3rd edn. Clemson: ICAI. Available at: <https://academicintegrity.org/resources/fundamental-values> (Accessed: 23 April 2026).

Krug, S. (2014) *Don't Make Me Think: A Common Sense Approach to Web Usability*. 3rd edn. Berkeley: New Riders. Available at: <https://sensible.com/dont-make-me-think/> (Accessed: 23 April 2026).

Appendix

Appendix A- Survey Questions and Results:

Section 1 of 2

StudyWest- Student Study Resource Sharing Survey

B *I* U ↗  

This short survey aims to understand how University of Westminster students currently share and access study materials, the challenges they face, and whether a dedicated study-sharing platform (like *StudyWest*) would be useful.

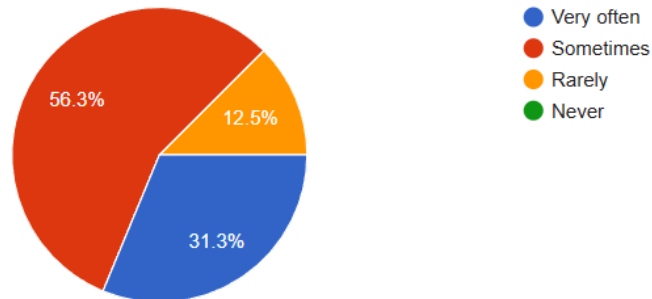
what is your course and year of study

Short answer text

How often do you share or receive study materials from classmates?

 [Copy chart](#)

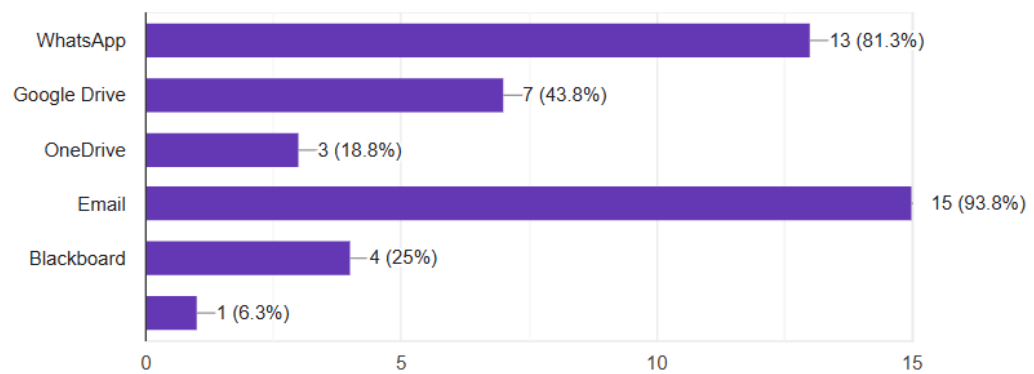
16 responses



How do you currently share study materials? *(Select all that apply)*

 [Copy chart](#)

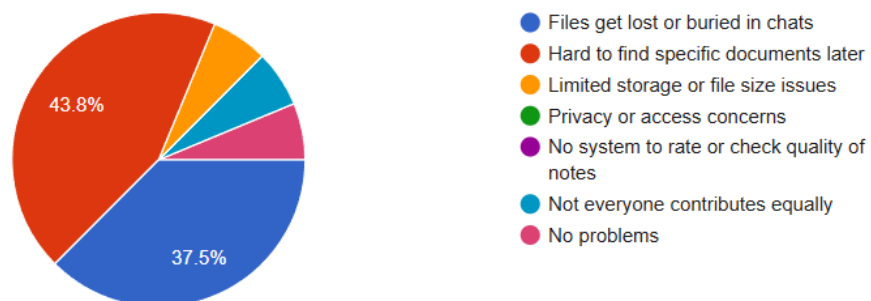
16 responses



What problems do you face when using these methods?

 [Copy chart](#)

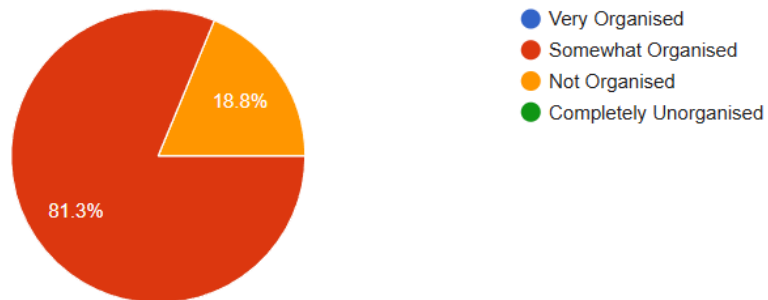
16 responses



How organised do you think shared study resources currently are within your course?

 [Copy chart](#)

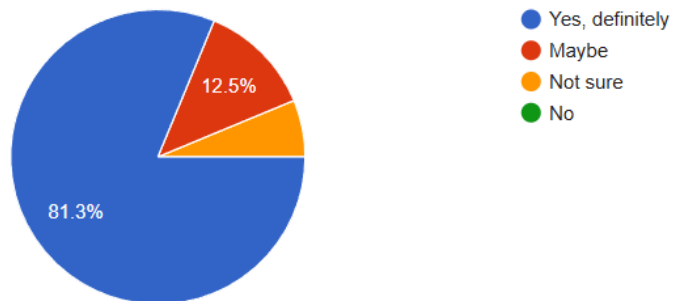
16 responses



Would you find it useful to have a single website or app where Westminster students can safely upload, share, and rate study materials for each module?

 [Copy chart](#)

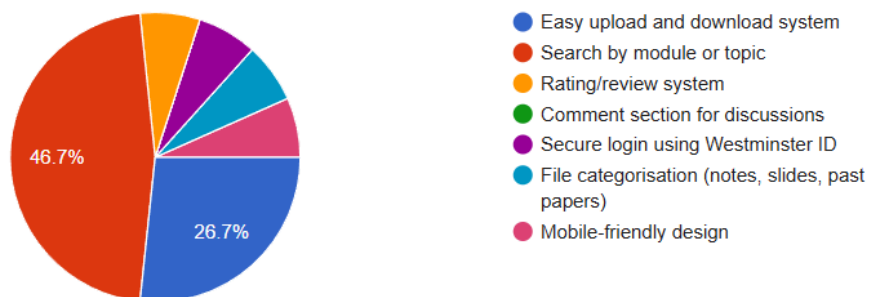
16 responses



What features would you most like to see in such a platform?

 [Copy chart](#)

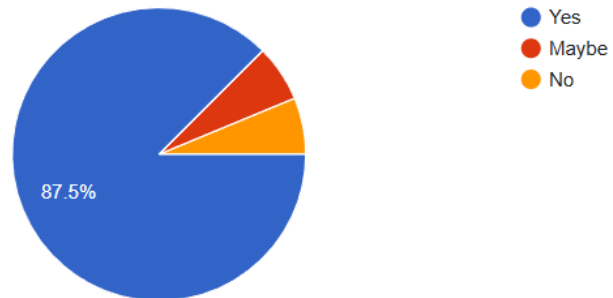
15 responses



Would you trust a University-exclusive platform (Westminster students only) to share and access notes securely?

 [Copy chart](#)

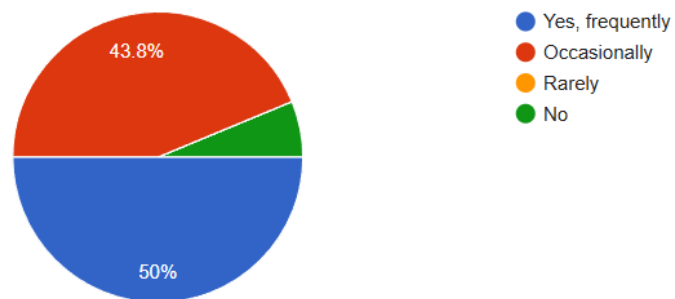
16 responses



Would you be willing to use such a website regularly during your course?

 [Copy chart](#)

16 responses



```
// types/auth.ts
export type UserRole = "student" | "administrator";
export interface AuthUser {
  uid: string;
  email: string;
  displayName: string;
  role: UserRole;
  verified: boolean;
}
```

```
// utils/validation.ts
const ALLOWED_DOMAIN = "@my.westminster.ac.uk";
export function isWestminsterEmail(email: string): boolean {
  return email.toLowerCase().trim().endsWith(ALLOWED_DOMAIN);
}
```

```
// components/ProtectedRoute.tsx
import { Navigate } from "react-router-dom";
interface ProtectedRouteProps {
  user: AuthUser | null;
  requiredRole?: UserRole;
  children: React.ReactNode;
}
export function ProtectedRoute({
  user,
  requiredRole,
  children,
}: ProtectedRouteProps) {
  if (!user || !user.verified) {
    return <Navigate to="/login" replace />;
  }
  if (requiredRole && user.role !== requiredRole) {
    return <Navigate to="/dashboard" replace />;
  }
  return <>{children}</>;
}
```

Appendix B shows the intended access-control structure used in the prototype. The submitted version applies role checks at routing level using mock authentication state. In production, these checks would need to be enforced server-side through Firebase Authentication and Firestore Security Rules.